CprE 381: Computer Organization and Assembly-Level Programming

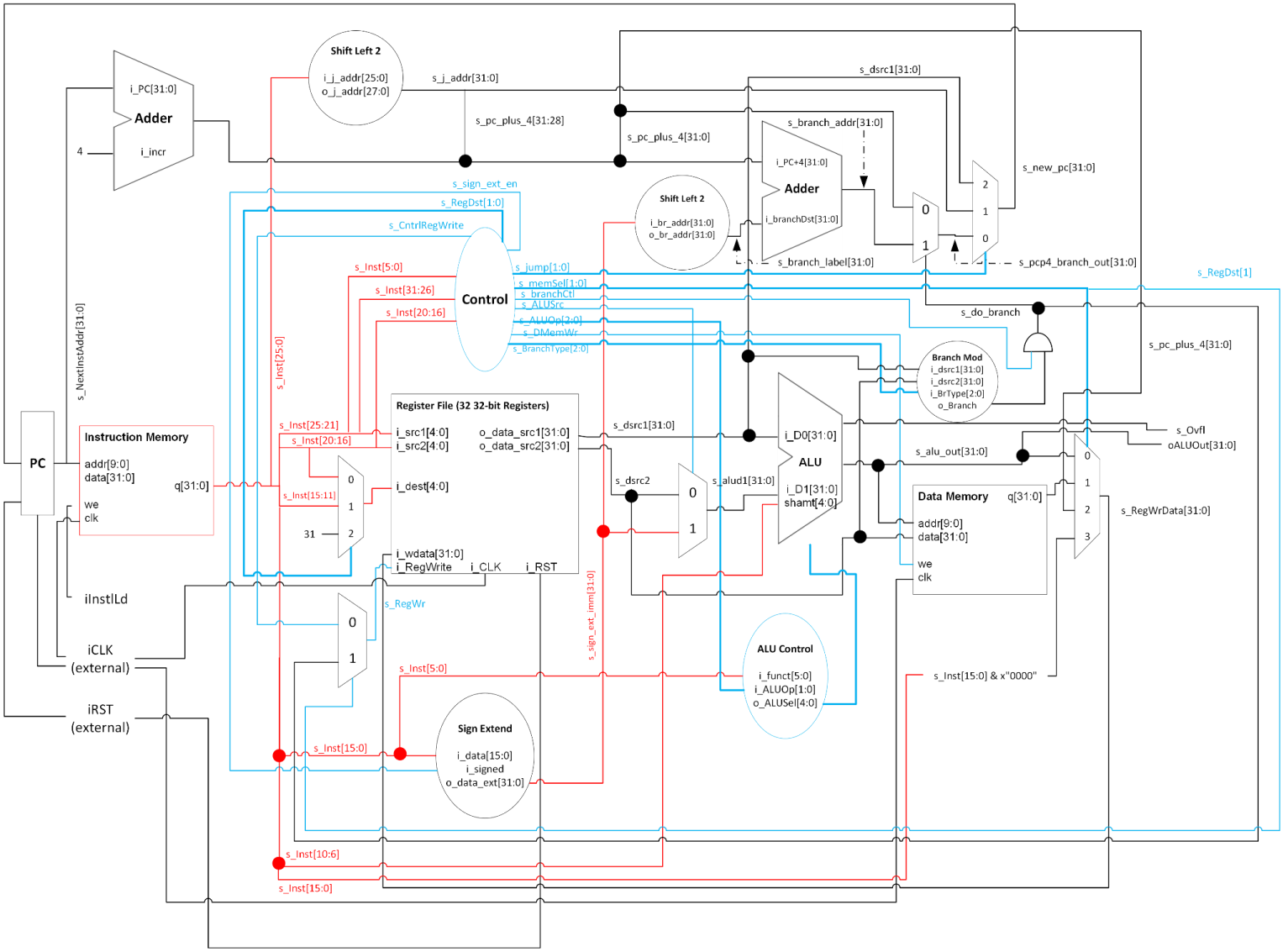
Project Part 1 Report

Team Members: Anthony Manschula
Henry Shires

Project Teams Group #: TermProj_3_02

Please see this folder for full-resolution waveform images.

[Part 1 (d)] Include your final MIPS processor schematic in your lab report.

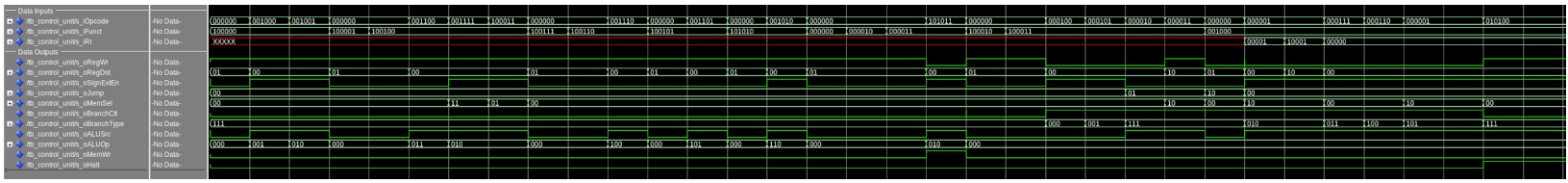


Our MIPS Single-Cycle Processor design

[Part 2 (a.i)] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The result should be an *N***M* table where each row corresponds to the output of the control logic module for a given instruction.

Spreadsheet: Processor Control Signals - TermProj_3_02

[Part 2 (a.ii)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).

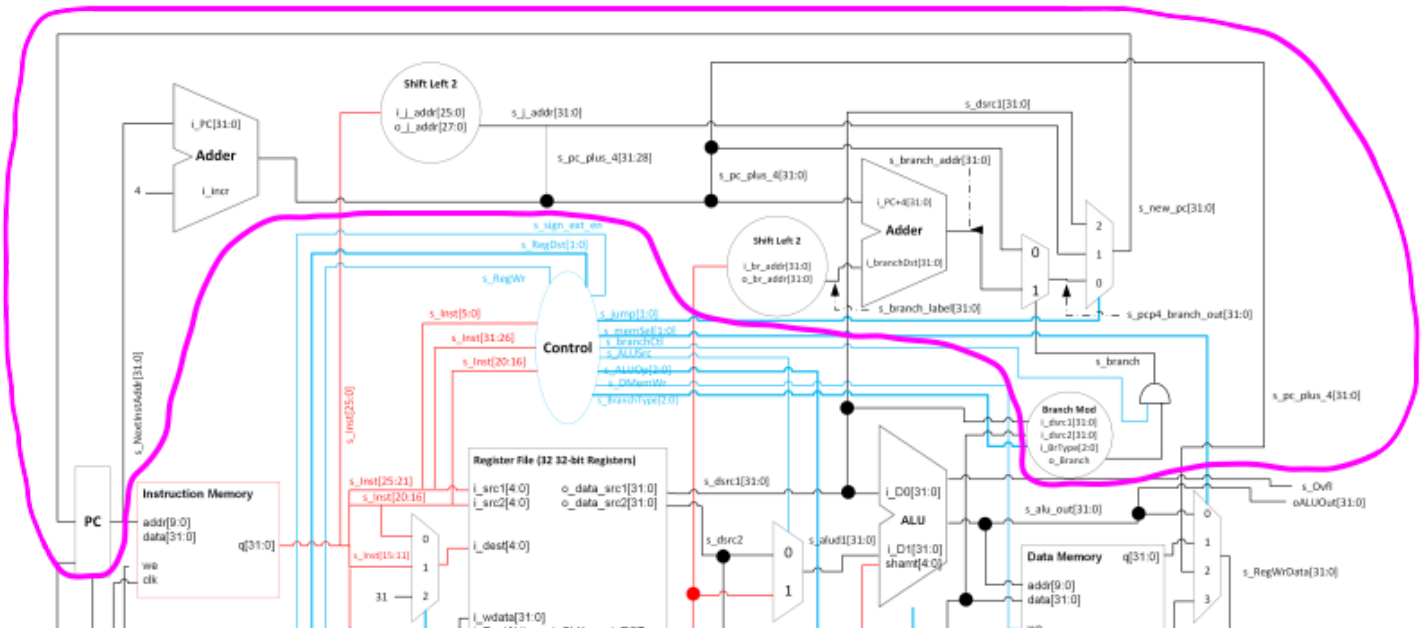


As programmed in the test bench, the QuestaSim waveform outputs all opcodes in the order listed in our control signals spreadsheet. The spreadsheet can then be easily read from top to bottom and compared with the waveform from left to right. Following this process, each control signal output matches its intended values for our control unit implementation. Note: The `s_iRt` signal is undefined until it is set in the test bench by the `bgez` instruction. Then its values appear as intended for the remaining branch instructions that require it.

[Part 2 (b.i)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

The instruction fetch logic must support several cases: Sequential execution (standard non-control flow instructions), conditionally setting the PC to an instruction address given by an immediate (branching) and occasionally setting $ra to the next instruction address in the case of *b*al*, and unconditionally setting the PC to the value of an immediate (jump) and setting $ra to the next instruction address in the case of a *jal*.

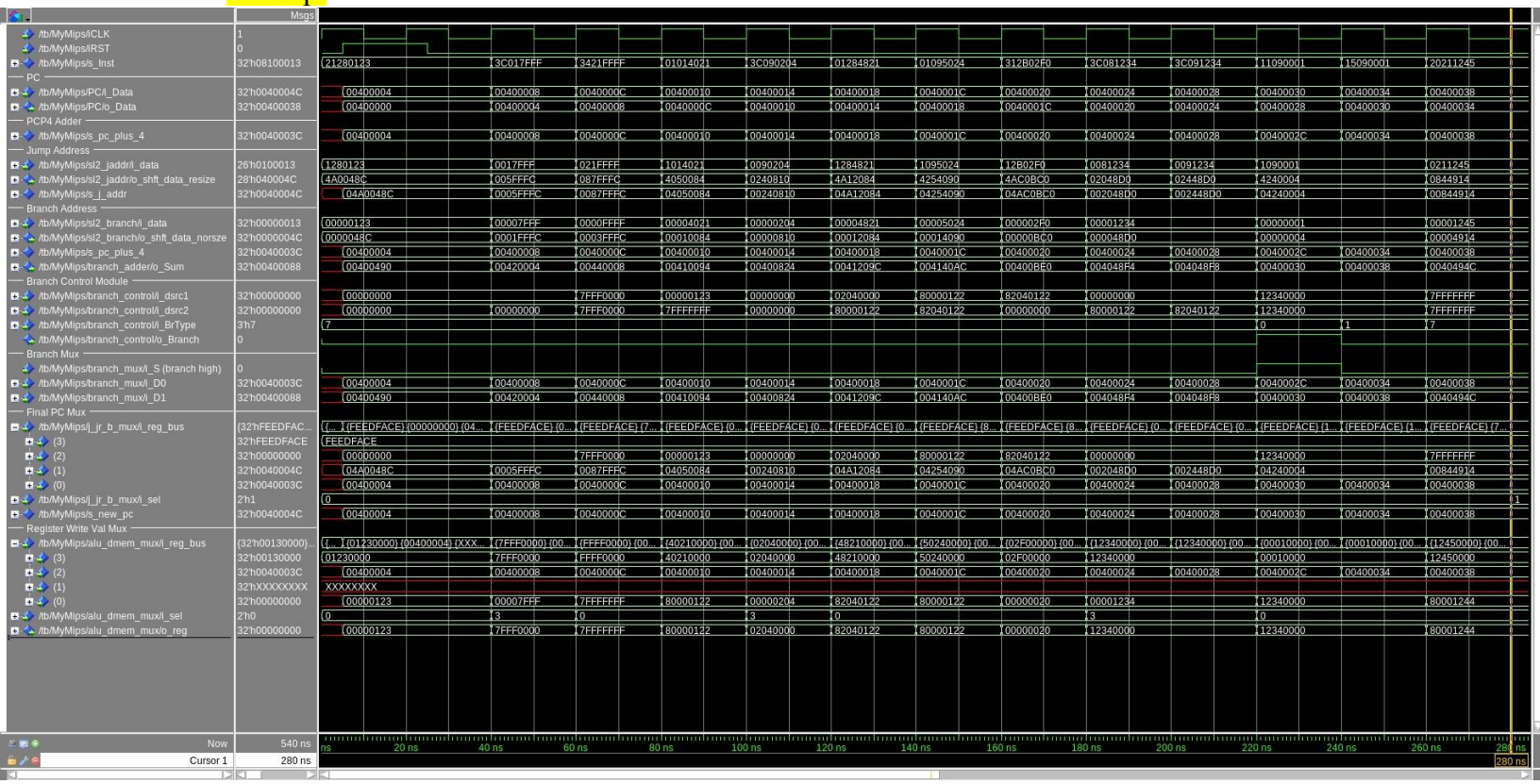
[Part 2 (b.ii)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What are additional control signals needed?

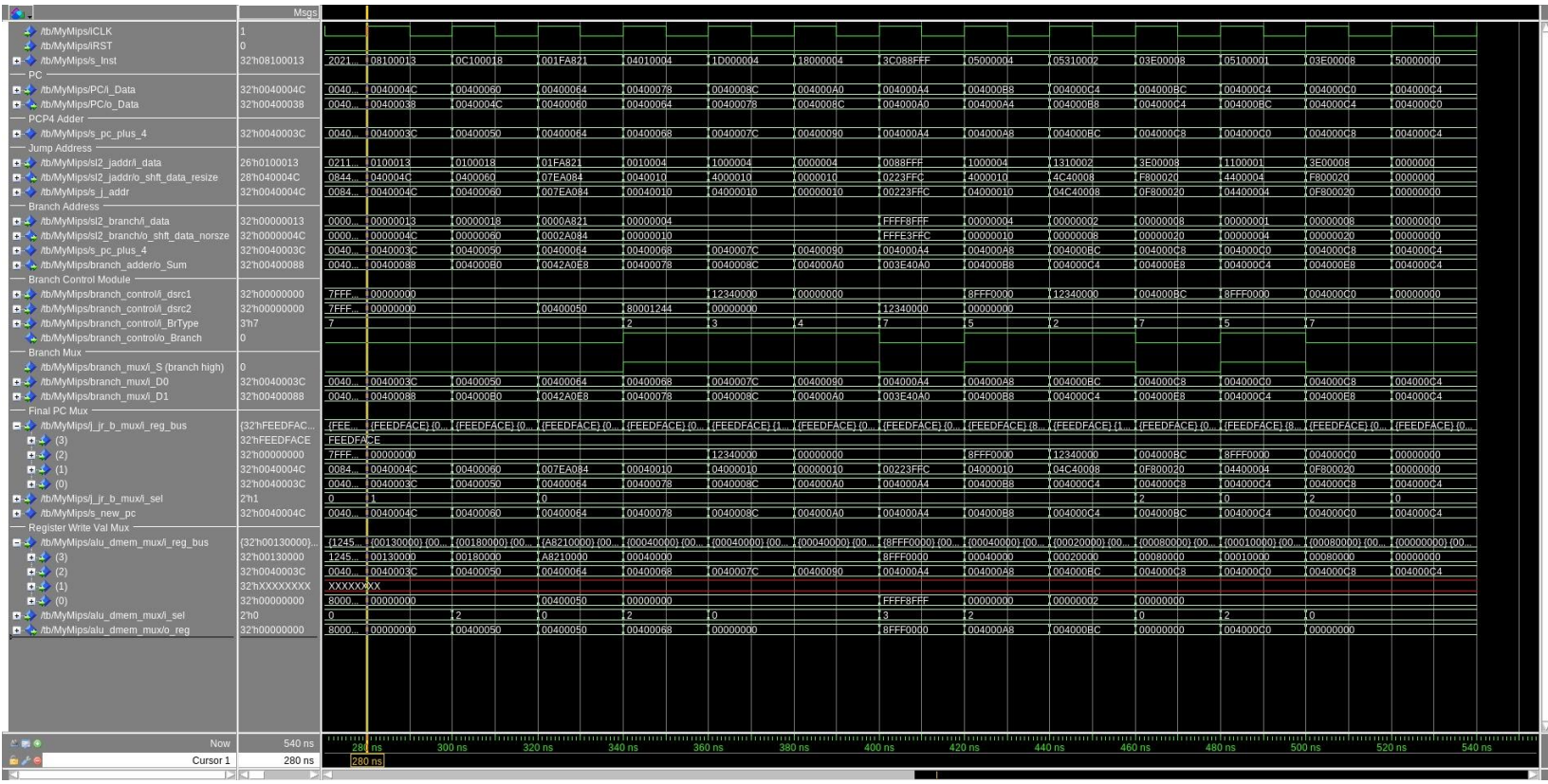


Additional control signals we elected to include in our design:

- *s_jump*: Controls whether the PC takes PC+4 (or branch address), a jump address specified by an immediate, or an address value specified in a register from `jr`.
- We also designed a standalone branch control module, which takes the *s_BranchType* signal generated from the control unit and determines whether a branch should be taken based on input data and the type of control flow being performed.

[Part 2 (b.iii)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to ensure it works as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.





- Sequential execution
    - The first 10 cycles are executing simple add, or, and, and lui instructions, as evidenced by s_Inst. It can be seen that the PC output data increments by 0x4 after the execution completes.
- Branching
    - The 11th cycle fetches a branch instruction. We can see that, although PC + 4 calculates to 0x0040002C, the input of the PC register is 0x00400030, which is the address of the branch. Moving down, we can ignore jump address since we are not jumping. We can see that the PC-relative branch address is calculated to be 0x4 (we are only branching over one instruction), and it is added PC+4 in order to obtain the final branch address. The branch control module compares the two inputs based on the i_BrType signal (0 =, 1 !=. 2 >= 0, 3 >0, 4 <= 0, 5 < 0) generated by the control module. In this case we are testing equality since BrType is 0, and since i_dsrc1 and i_dsrc2 are equal, the branch output is asserted. This same process applies for any of the other branch instructions. The output of the branch module controls the select line for the branch mux. Since it is 1, we choose to take the value of the branch address (calculated in section “Branch address”) instead of PC+4. We then move on to the final step, which is choosing the data to send back to the PC. The select line of this multiplexer also comes from the control unit, and in this case, a select value of 2b00 will result in the data from the branch mux (in the case of a branch this will be the branch address) being sent to the PC input via s_new_pc. There is also another case, which are the *b*al* instructions. These must set $31 to the value of PC+4 when the branch is taken. If we jump to instruction 22 in the program (a BGEZAL), we can see that in addition to the aforementioned steps, we also set the select signal for the register write value mux to 2b10, which allows PC+4 to pass through to the write data port of the register file. A select signal also sets the destination register to a hardcoded decimal 31 (not shown).
- Jumping
    - The 14th instruction is a jump (at 280ns). We see that the input of the PC is 0x0040004C, which indicates that the jump address was properly calculated. Indeed, in the “Jump address” section, we can see that the input data from the instruction is properly shifted left two bits and resized. Skipping down to the “Final PC Mux” section, the calculated branch address is seen on the input bus, and the select signal coming from the control module is 1, which indicates we should send the jump address to the PC input. In the 15th cycle (300ns), we perform a *jal*. All of the relevant information just mentioned still applies, and we can see that we end up with a proper jump address calculated and sent to the PC. However, since we are linking, we also need to write PC+4 to $31. We can see that the value of the register write value mux select signal is 2, which indicates we should pass PC+4 to the reg write port. The last jump case is *jr*, which is straightforward. The 25th instruction (500ns) is performing a *jr $31* instruction to return from a *jal.* We simply set the select signal for

the Final PC Mux to 2b10, and this passes through the value of data source 1 from the register file.

[Part 2 (c.i.1)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does MIPS not have a `sla` instruction?

Logical shifts do not account for the value of the bit at either extreme of the signal (i.e., bit 0 or bit 31) when shifting. They simply shift bits left or right and insert 0s at the opposite end of the direction of the shift. Arithmetic right shift preserves the sign bit's value by shifting the sign bit's value. MIPS does not have a `sla` instruction because the LSB doesn't represent much useful information in the context of an arithmetic shift.

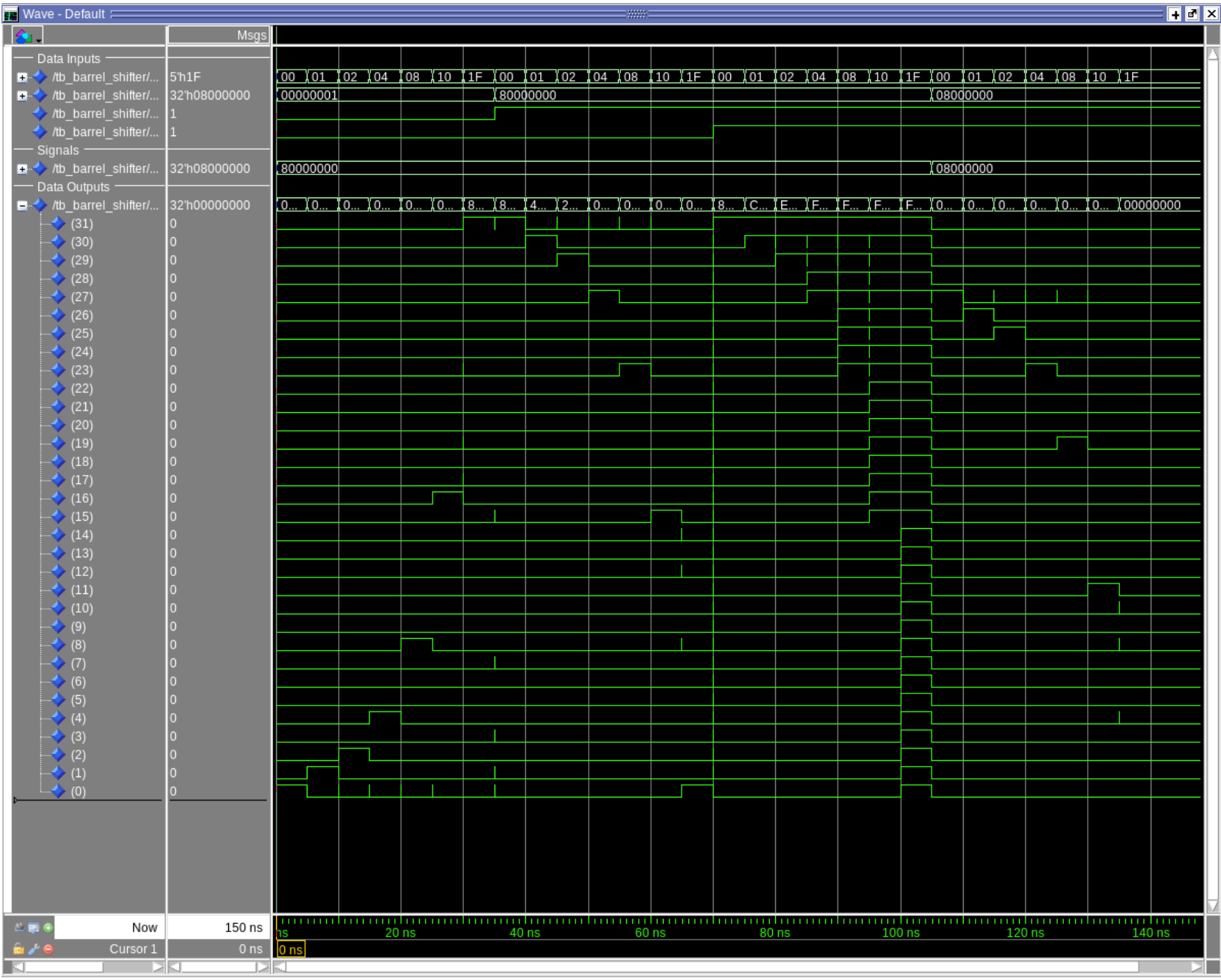
[Part 2 (c.i.2)] In your write-up, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.

In logical shifting, bits left over after shifting are always set to 0. For the left shift, this means the bits to the right of the shifted bits are set to 0, and vice versa for the right shift. In arithmetic shifting, instead of replacing the leftover bits with 0s, they are replaced with the sign of the original binary number, in this case, using 2s complement, the most significant bit's value (MSB). Our VHDL implementation uses 2:1 multiplexers to manipulate each bit of the 32-bit input vector *`i_shsrc`*`[31:0]`. For each bit of the 5-bit shift amount vector *`i_shamt`*`[4:0]`, a row of 32 2-1 MUXs with the corresponding shift amount bit as a selection for all is used to shift the bits X digits left or right. Suppose X is greater than the remaining bits available to shift. In that case, the MUX will use a 0 or 1 to assert on the shifted bit instead, with a 0 for logical operations and a 1 for arithmetic operations. The signal propagates through the X = 16-bit row, 8-bit row, 4-bit row, 2-bit row, and 1-bit row, a total of five rows, each matching a shift amount bit (1 or a 0). The outputted vector with the correct shift applied should appear at the end of the multiplexer array.

[Part 2 (c.i.3)] In your write-up, explain how the right barrel shifter above can be enhanced to support left-shifting operations.

Since the right barrel shift output bits directly manipulate *`i_shsrc`* in order, *`i_shsrc`* can be split up into bits 0-31 and inputted into the first row of 2-1 MUXs from left to right, shifting right based on the value of *`i_shamt`*. With left shifting, the current vector direction will result in an incorrect shift. While seemingly complicated, the right barrel shift can be easily modified to reverse the vector bits using an additional set of MUXs, where bit 31 of the original input is now located at bit 0 and vice versa, continuing until the entire vector has been flipped. This row of MUXs utilizes a separate select *`i_shdir`* where 0 corresponds to the left shift and 1 to the right shift. Each bit is either left alone or reversed with its opposite position (0 and 31, 1 and 30, etc.) based on the select value. Then the exact same right barrel shifter can be used with the reversed input for a left shift. The reverse vector process is repeated to undo the reversal of bits at the end of the barrel shifter to ensure the original input with the correct shift is outputted.

[Part 2 (c.i.4)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.



In our QuestaSim test bench, four common cases are tested from left to right. From 0-35, ns is `sll` with *`i_shsrc`* = 0x00000001, 35-70 ns `srl` with *`i_shsrc`* = 0x80000000, 70-105ns `sra` with *`i_shsrc`* = 0x80000000, and 105-140 ns `sra` with *`i_shsrc`* = 0x08000000. These specific input values help visualize the shift that occurs one after another. For cases 3 and 4, case 3 is run with the MSB of the input = 1 and case 4 with the MSB = 0 to ensure the arithmetic operation follows the sign of the vector. In all four cases, the '1' value on the input is seen shifted 1, 2, 4, 8, 16, and 31 bits as intended. Case

1 shifts it from LSB to MSB (left), case 2 from MSB to LSB (right), case 3 from MSB to LSB while tracing the sign of 1, and case 4 from MSB to LSB while tracing the sign of 0.

[Part 2 (c.ii.1)] In your write-up, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

- MIPS PC
  - Design: We needed a component that was similar in behavior to a register but with a MIPS-compliant reset value. We followed the n-bit register design closely and included a reset value of 0x00400000.
- Shift Left 2
  - We needed a shift left 2 unit that could either take the form of an n-bit and n+2-bit input/output or an n-bit input/output. The choice was made to use a combination of dataflow and behavioral VHDL in order to create something that was reusable for both the jump address shift, as well as the branch address shift.
- Sign extender
  - We wanted an approach that was straightforward, so we opted to use a select statement in combination with some dataflow-style VHDL to map the input signal to the lower part of the output, and fill the upper part of the output with either 0's or the sign bit, depending the value of the sign extension enable control.
- ALU Control
  - The ALU control module needs to tell the ALU which arithmetic operation to perform based on the *funct* field of the instruction, as well as the ALUOp control output. The ALUSel output is selected for an R-type instruction when ALUOp is '000' and a matching *funct* is found on the input. The value of ALUOp may also be the same for some non-R-type instructions, such as `add` and `addi`.
- Branch Module
  - We quickly realized that the "zero" output of the ALU would not have been sufficient to support the required branch instructions and that trying to shoe-horn support into the ALU would have been messy and error-prone. We instead opted to add an entirely new module that outputs a "do branch" signal based on the two register outputs and a new control signal called BranchType, which is generated by the control and is unique to each type of branch instruction.

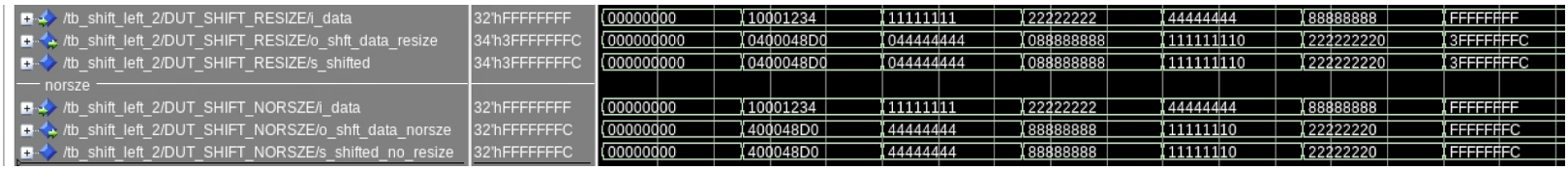
[Part 2 (c.ii.2)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

MIPS PC



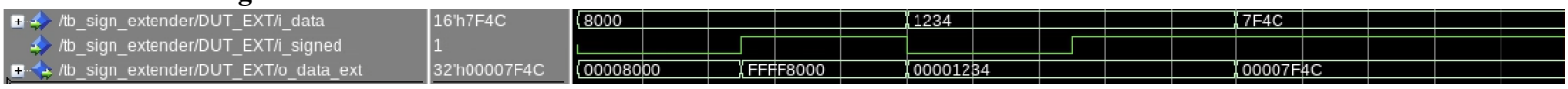
- At its core, the MIPS PC is just a register with a custom reset value, and we've already validated the register design in previous labs. Therefore, we tested the reset functionality by making sure reset assigned the correct value, and that setting the input while reset is asserted did not affect the output in the first two clock cycles. From there, the register operates as expected with the reset signal not asserted.

Shift Left 2



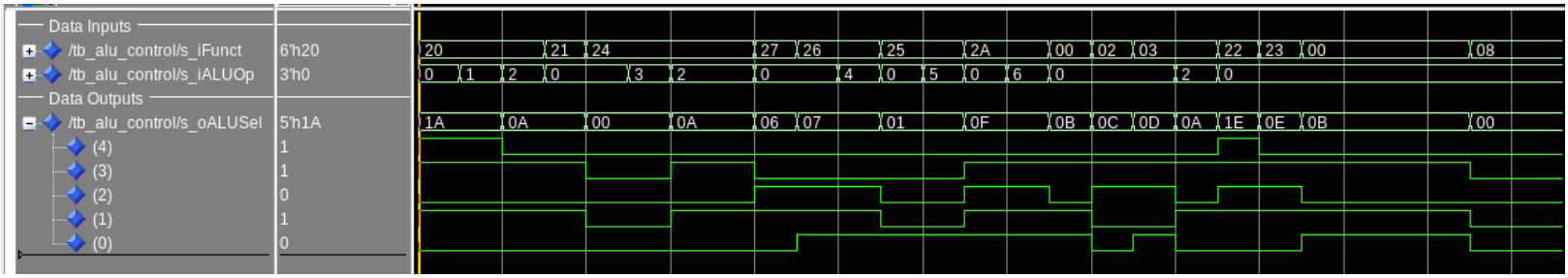
- The shift left 2 module shifts the input left by two bits, and optionally increases the output width by two bits as well. Width increase is important because it is used in the jump address calculation part of the datapath. As it can be seen in the waveform, the input bits are shifted left two positions and the vacated positions are filled in with zeroes. Additionally, it can be seen that the output is resized correctly in the case of the top signal, and that it works properly for all 0's and all 1's input.

Sign Extender



- The sign extender simply maps the 16-bit input signal into the lower 16 bits of the output, and then concatenates either 0's or the value of input[15] onto output[31:16] when i_signed is 0 or 1, respectively. Due to the nature of this implementation, it will either work, or it will not work. There won't be some input values that it works for and some that it doesn't, so extensive testing would be redundant. It can be seen that for s_signed = 0, 0's are concatenated regardless of the value of input[15]. When s_signed = 1, the value of bit[15] is concatenated, as seen in the outputs for input 0x8000, as well as 0x1234 and 0x7F4C.

ALU Control



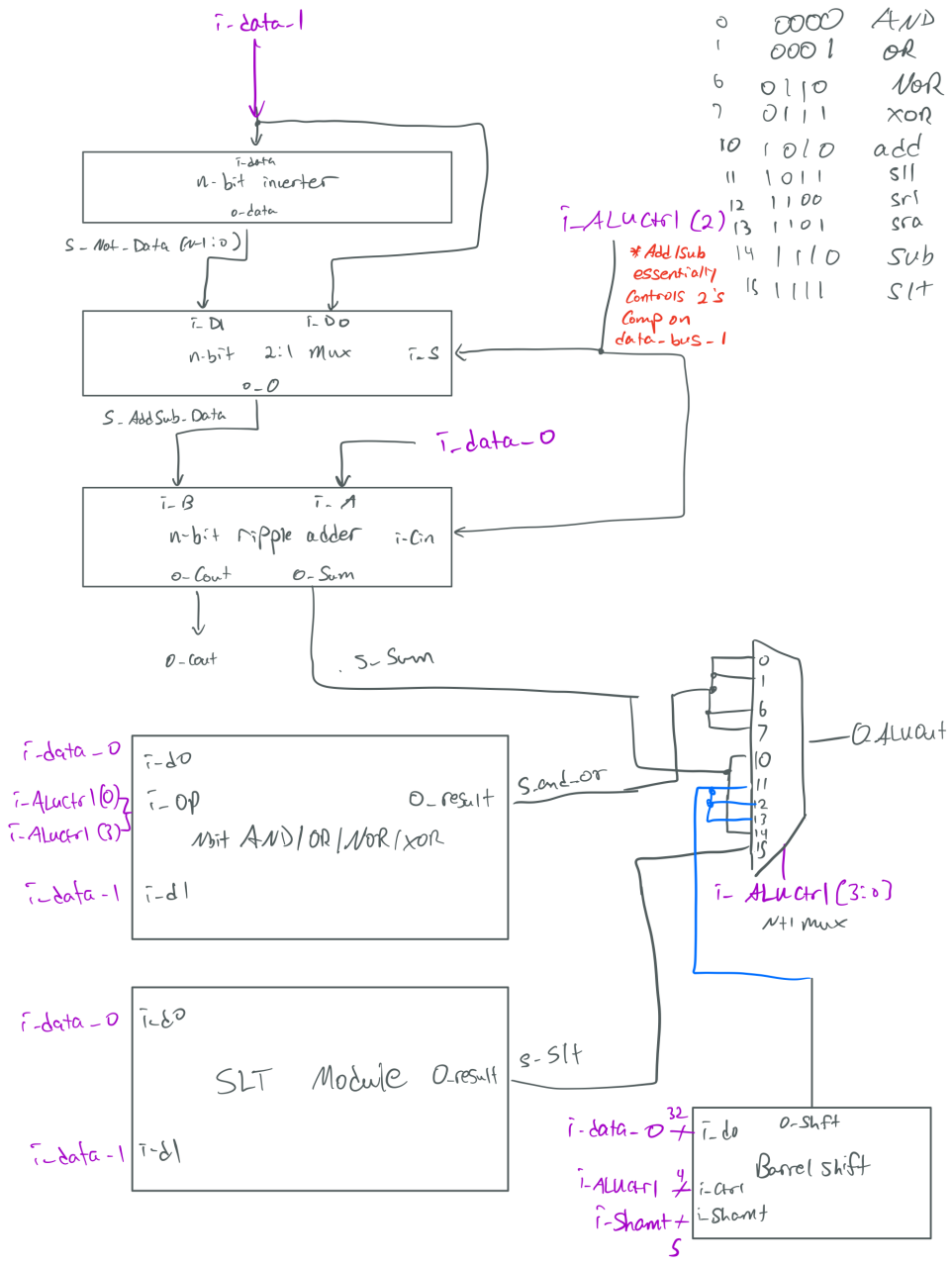
- The ALU Control module outputs ALUSel to the ALU in order to tell it which arithmetic operation to perform. Some ALU operations are selected based purely on the *funct* field and *ALUOp == b000 (meaning an R-type),* whereas some ALU operations are selected with a funct and zero ALUOp or a non-zero ALUOp. This is for ALU operations that might have multiple associated instructions, such as addi and add (at the end of the day they are both adding, although one is I-Type and the other is R-Type). We can see that, for ALU operations with both a non-zero ALUOp as well as a zero ALUOp and a funct, we test both cases to ensure we get the correct ALUSel from the spreadsheet. Otherwise, we just compare the ALUSel output to the expected ALUOp on the spreadsheet.

Branch Module



- We test each branch type input with data that will cause the branch output to assert and data that will not cause the branch output to assert. We start with branch equal. The data is not the same, so it doesn't branch, but we see the output go high when it is. For a branch not equal (BrType = 1), it is the reverse of the previous condition. We test branch on greater than or equal to 0 (BrType = 2) for a negative, 0, and a positive, all giving the expected results. We follow that with a branch greater than 0 (BrType = 3) using a 0 input and a positive, also giving what we expect to see. We repeat the greater-than-equal test data, but instead, we choose a branch less than equal to 0 for the branch type (4), and the expected output is seen. The last branch type is less than 0 (5), and with 0 data, we expect to not branch, and with negative data, we do branch, which is what happens in reality.

[Part 2 (c.iii)] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: how is Overflow calculated? How is Zero calculated? How is `slt` implemented?

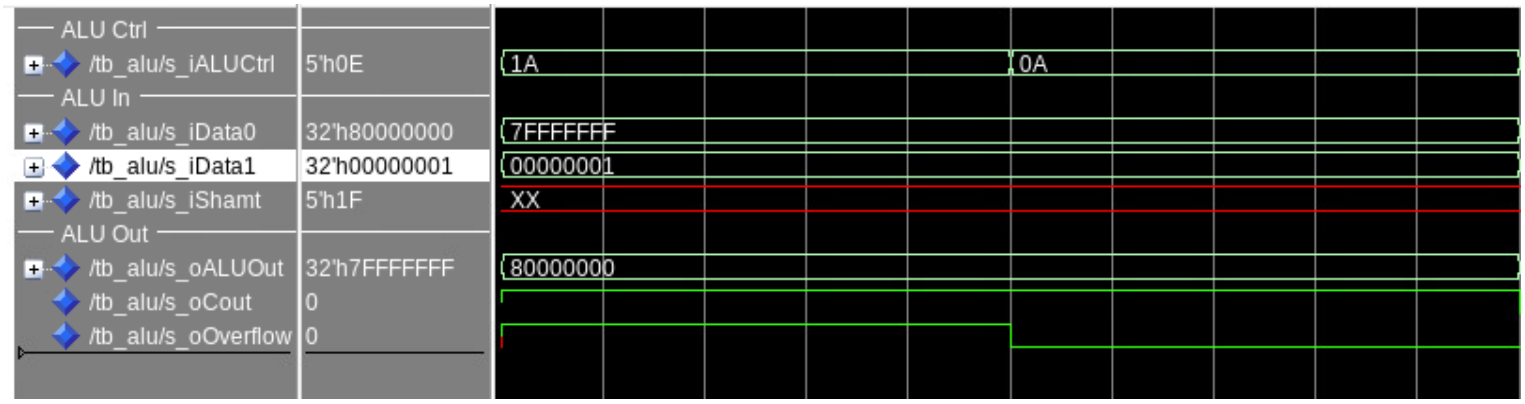


Overflow occurs when the input signals have the same sign, and the output has the opposite sign when performing addition. When performing subtraction, overflow occurs if the signs of the inputs are different and the output has the same sign as the second data source. The overflow signal is only asserted when an instruction deals with data represented in a signed form. We do not calculate a zero signal since it was originally used for branching, but since we have a standalone branch module, it is redundant. SLT is

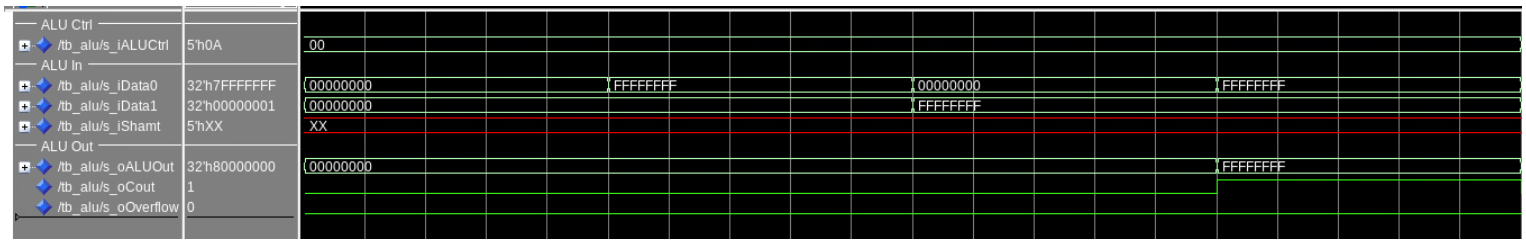
implemented via a simple behavioral module that compares the two (signed) inputs and asserts an output if the first input is less than the second.

[Part 2 (c.v)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

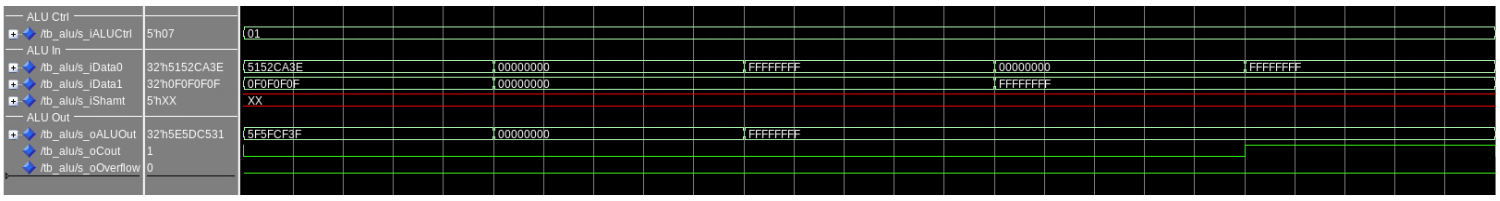
In our test bench, our ALU is operating as expected. Addition operations are either incrementing the number or wrapping it around if it reaches a result requiring more than 32 bits to represent. This is also the case for subtraction, but with decrementing the number. AND and OR also behave as expected, following the respective truth tables outputs.



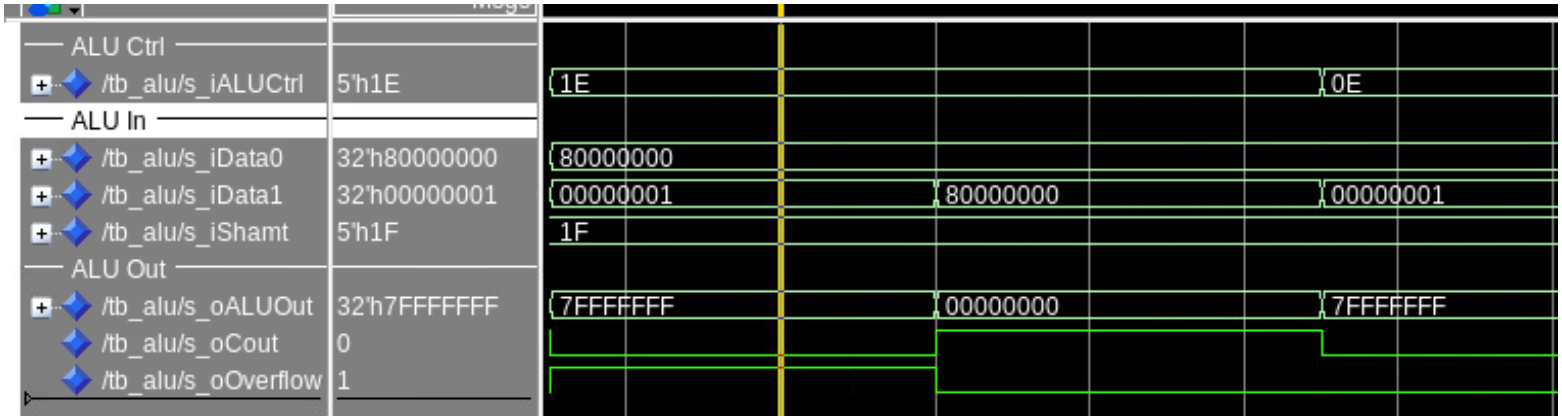
Add (signed with overflow then unsigned)



AND common cases from the truth table



OR common cases from the truth table + edge case

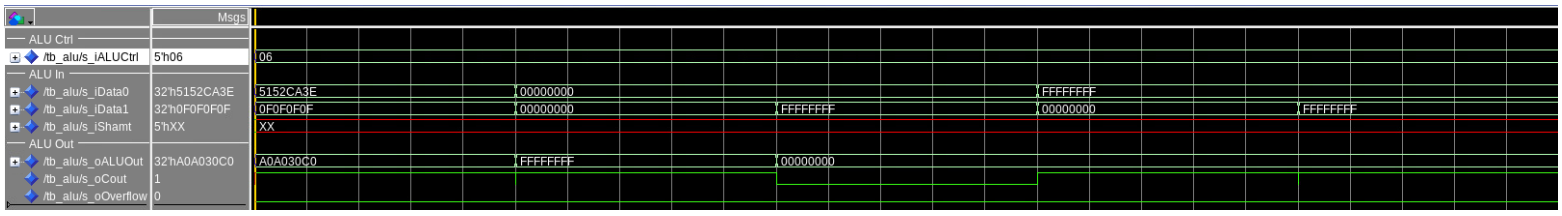


Sub (signed with overflow then unsigned)

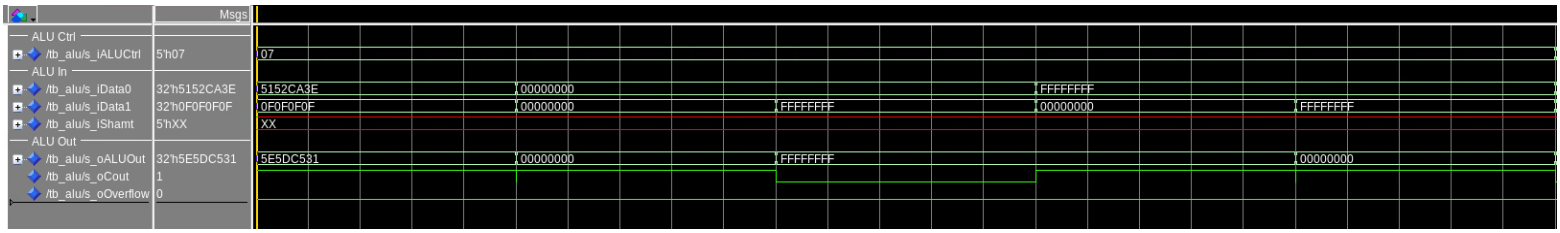
[Part 2 (c.viii)] Justify why your test plan is comprehensive. Include waveforms that demonstrate your test program's functioning.

To ensure our testbench is comprehensive, we've included multiple common and edge cases for each instruction where applicable. For the addition operation, we've included a normal, signed case, a normal unsigned case, and an overflow signed case. This is the same for the subtraction operation. For AND, we incorporated a bit-addressable example and all 0s or all 1s in the vector and changed when they were used for four cases. We replicated the alternation of using 0x00000000 and 0xFFFFFFFF for AND and other logical operations (NOR, XOR, and OR). These cases matched the general idea of their truth tables but at a byte level, while our edge case addressed individual bits for each. For set less than operations, we included changes in the sign of each input, alternating between two positives, one positive and one negative (and vice versa), and both negatives. Finally, for sll, srl, and sra, each case has a single 1 on the input and shifts by the next bit (0, 1, 2, 4, 8, 16, and 31) during each succeeding case. This produces the same output from the barrel shifter test bench to ensure an easy match between the two outputs.

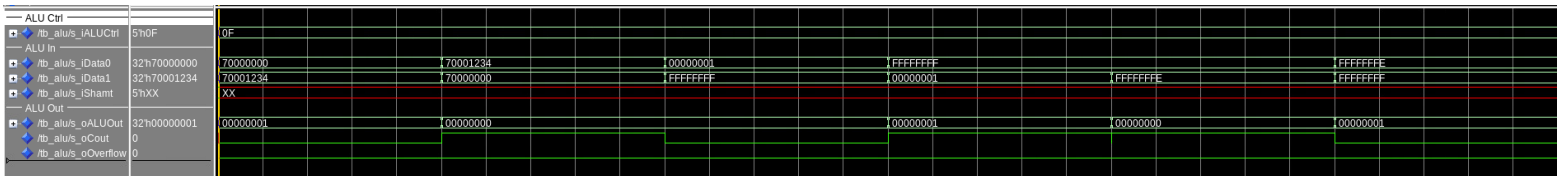
Overflow is asserting only on signed operations and asserting only when overflow occurs.



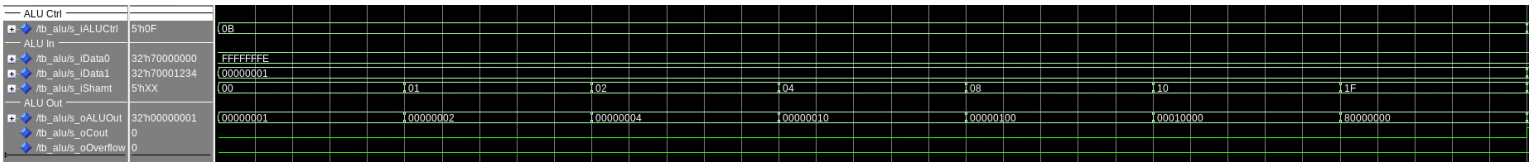
NOR



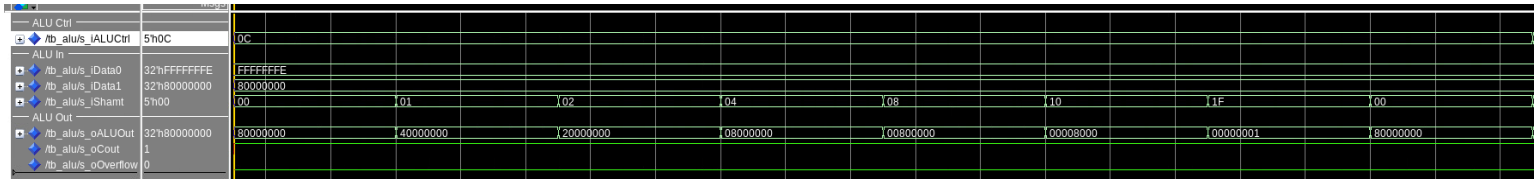
XOR



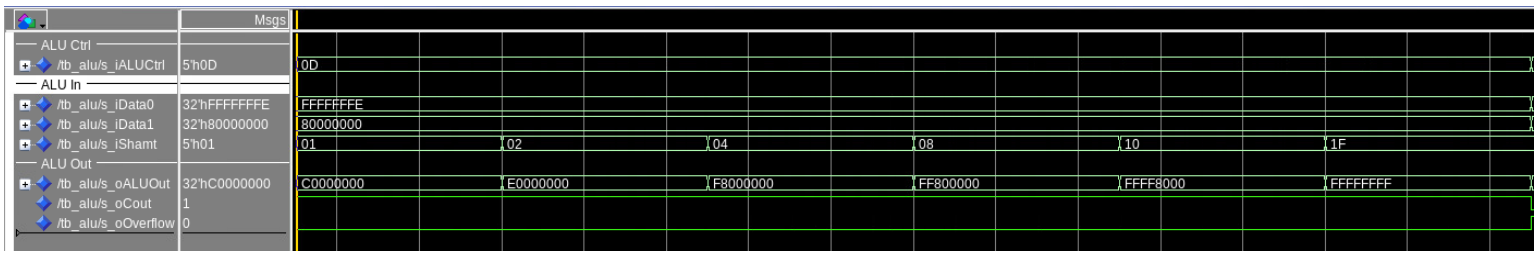
SLT



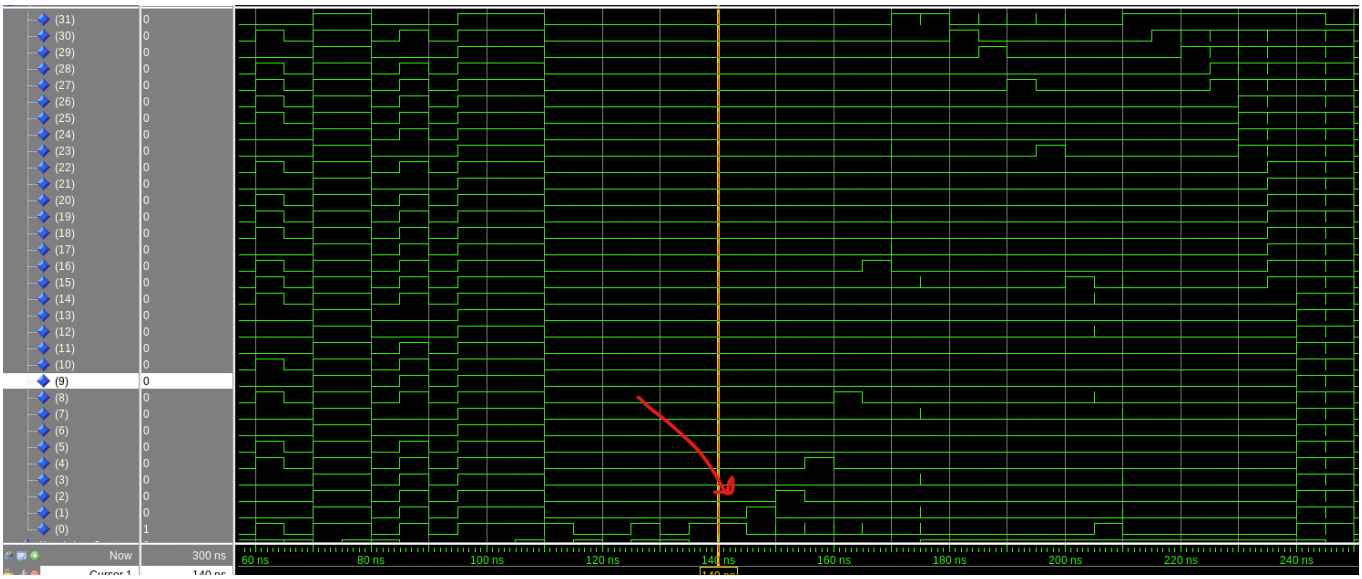
SLL



SRL



SRA

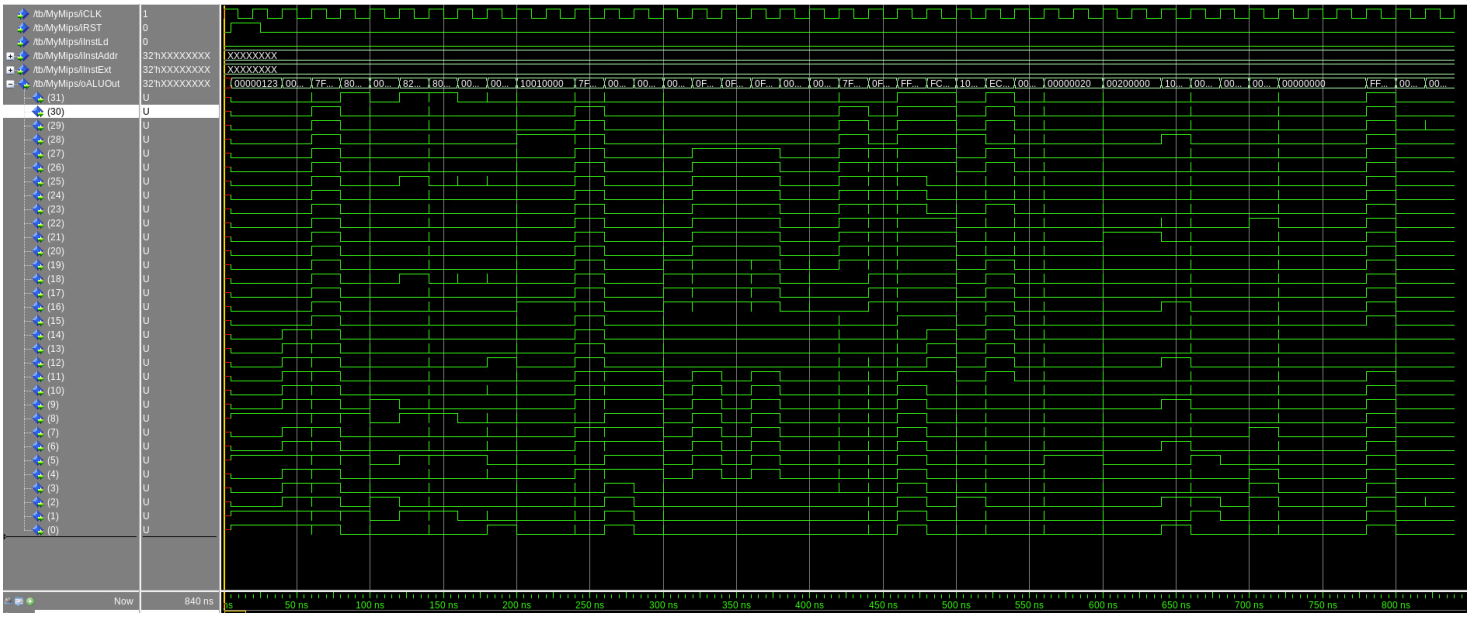


Reproduced Barrel Shifter Output (the arrow is the start of sll tests). The stack of waveforms matches the barrel shifter test bench earlier in this report.

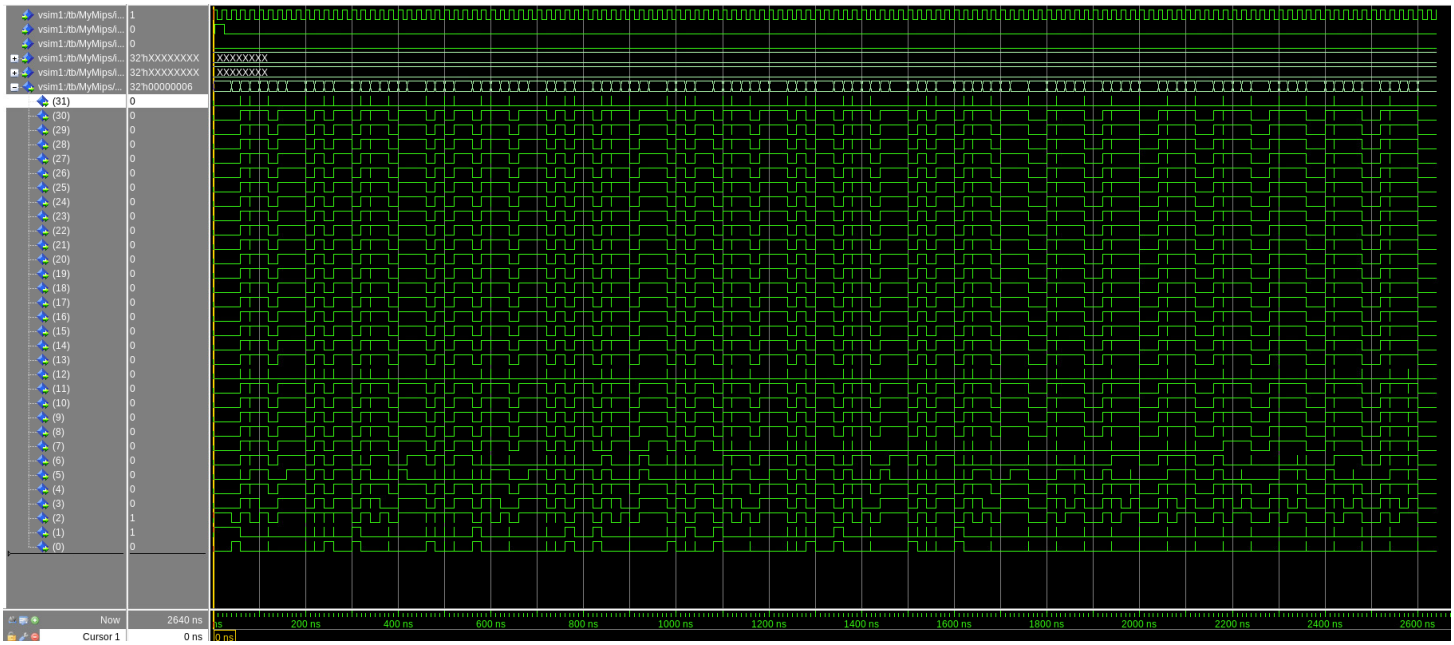
[Part 3] In your writeup, show the QuestaSim output for each of the following tests and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

In all three waveforms, with each instruction usually resulted in a register or memory update. Each waveform shows the output of the ALU, which should correspond with any register changes. For example, in the base_test program, register $8 is written $t1 + 0x123, which is populated on the waveform from the ALU.
Additionally, in cf_test, $4 is written 0 + 6, which populates on its waveform as 0x6, which is expected. Finally, the Bubblesort algorithm had a very large execution time and generated thousands of nanoseconds of signal propagations due to its recursive nature and the intensity of the sorting algorithm. After ~20,000 nanoseconds, Bubblesort completes and calls a halt, which sets the ALU output to 0 to exit the program seen on the waveform. Additionally, the address of the array data label can be seen computed after several previous instructions, which are used to reference the items to be sorted.

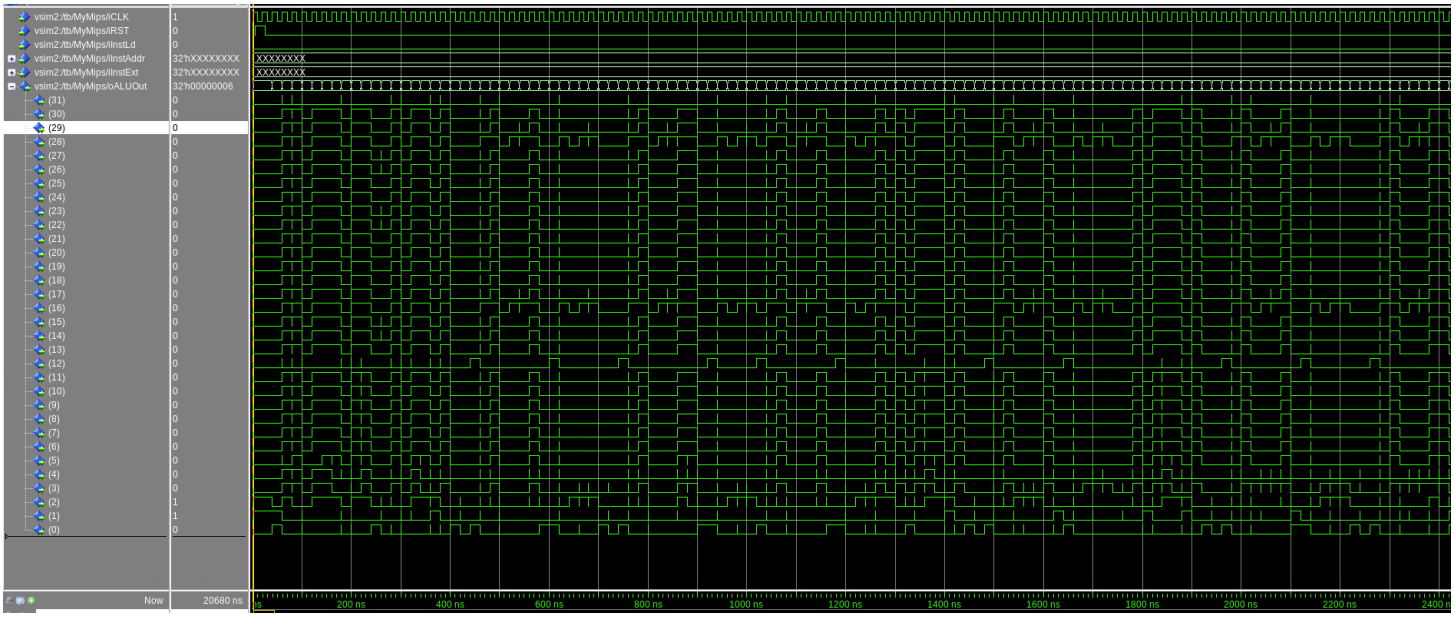
[Part 3 (a)] Create a test application that uses every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle, while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.



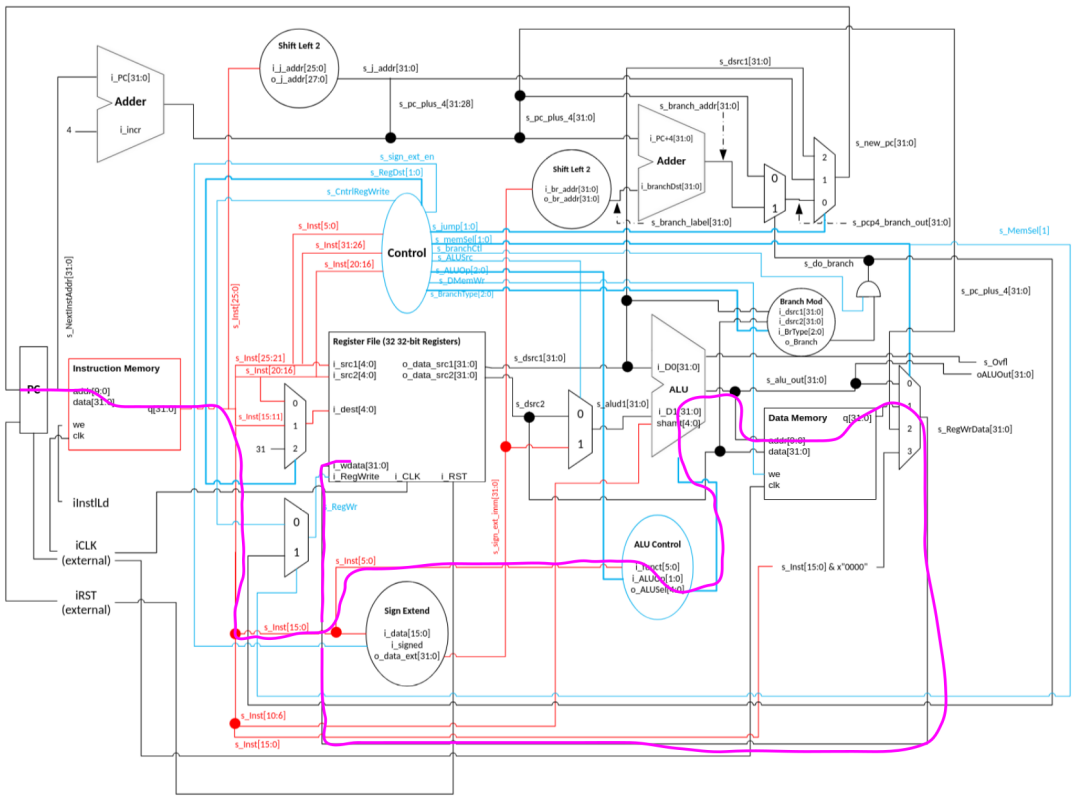
[Part 3 (b)] Create and test an application that uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.



[Part 3 (c)] Create and test an application that sorts an array with *N* elements using the BubbleSort algorithm (link). Name this file Proj1_bubblesort.s



[Part 4] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?



- The max frequency is 22.49MHz.
- The critical path is PC > IMEM > ALU Control > ALU > DMEM > DMEM Multiplexer > Write to reg.
- It would probably make the most sense to optimize the ALU Control since there are 3.5ns of latency alone in the first step of the path through it. The barrel shifter also adds quite a bit of latency, so it might be interesting to see what a non-structural implementation could result in.

CprE 381: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Anthony Manschula
Henry Shires

Project Teams Group #: TermProj_3_02

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

IF/ID: PC+4, Instruction
ID/EX:

- Old: PC+4
- New: new_PC, do_branch, CtrlRegWrite, RegDest, jump, memSel, ALUSrc, ALUOp, DMemWrite, halt, dsrc1, dsrc2, sign_ext_imm, Inst[20:16], Inst[15:11], Inst[5:0], Inst[15:0], Inst[10:6]

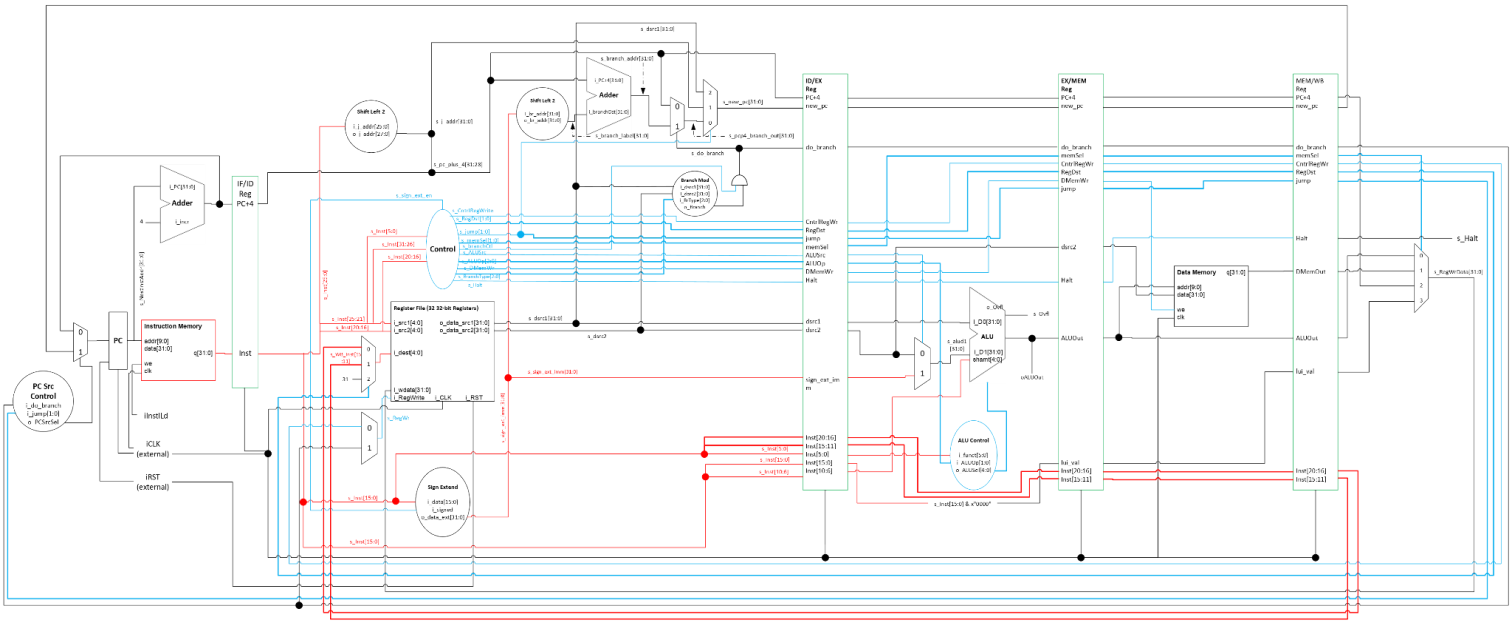
EX/MEM:

- Old: PC+4, new_PC, do_branch, CtrlRegWrite, RegDst, DMemWrite, jump, memSel, halt, dsrc2, Inst[20:16], Inst[15:11]
- New: lui_val, ALUOut

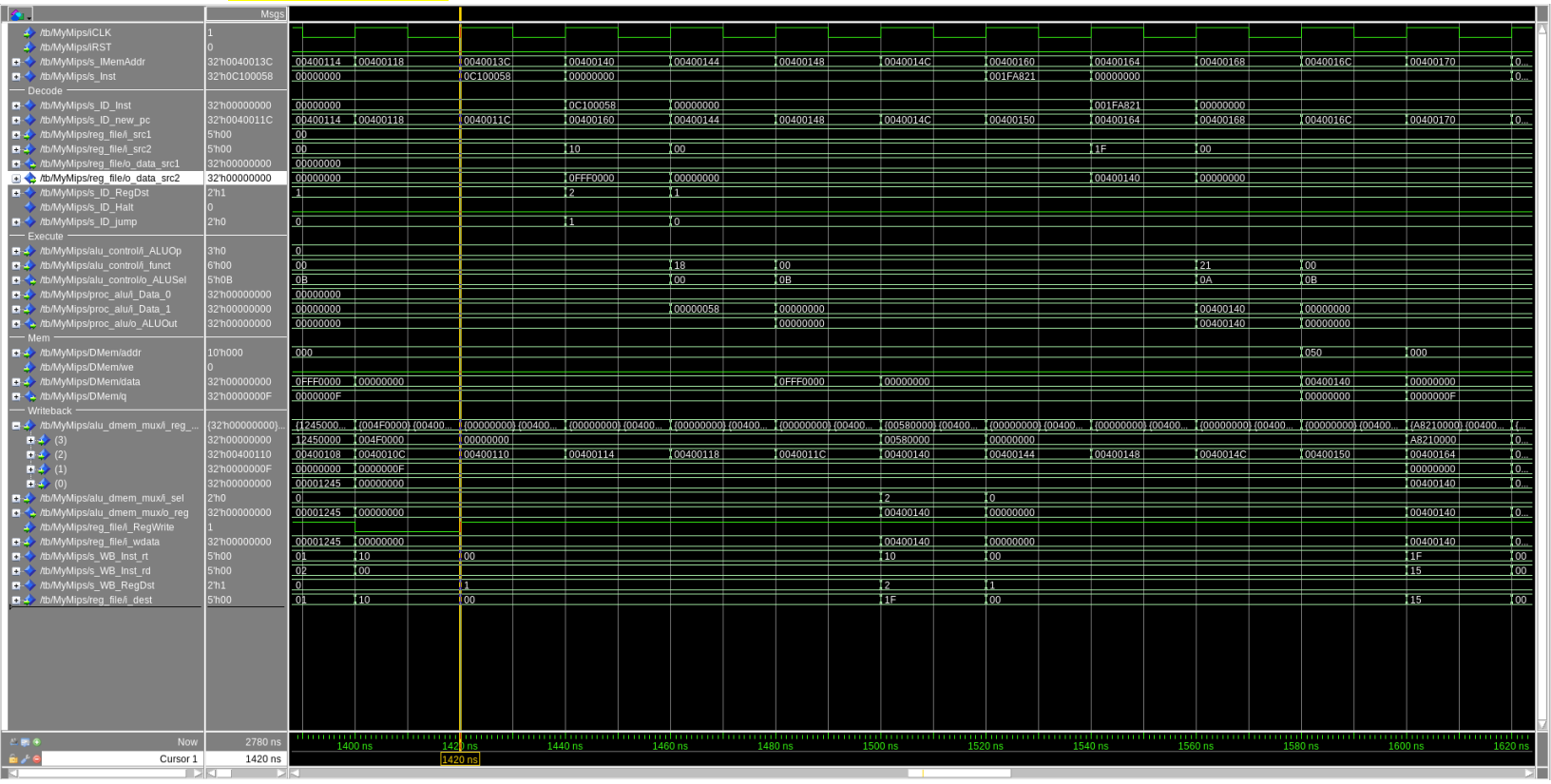
MEM/WB

- Old: PC+4, new_pc, do_branch, memSel, CtrlRegWrite, RegDst, jump, halt, ALUOut, lui_val, Inst[20:16], Inst[15:11]
- New: DMemOut, s_RegWrData

[1.b.ii] High-level schematic drawing of the interconnection between components.

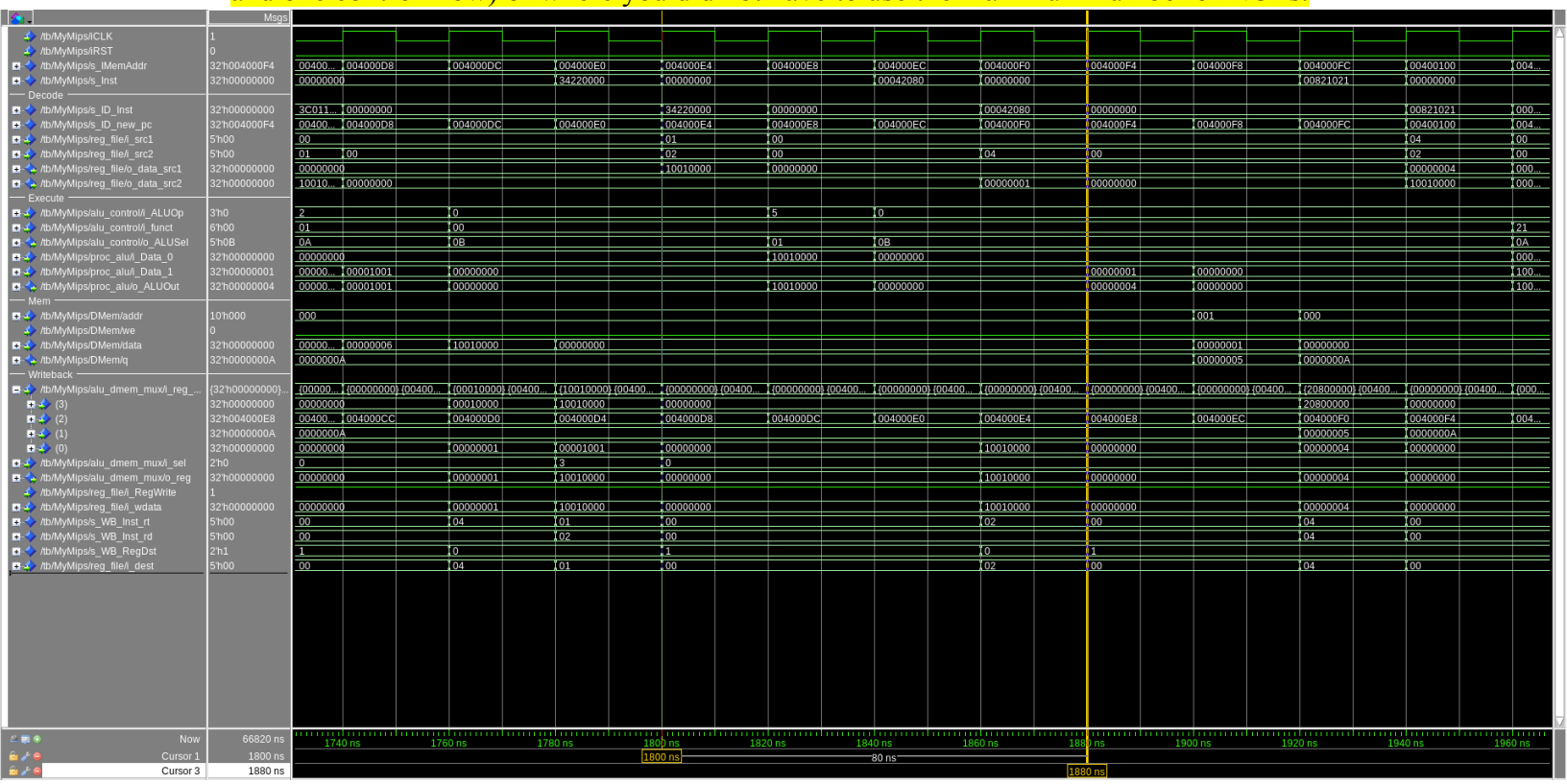


[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.



- The cursor is placed at the point at which a jal instruction is decoded. Over the next four clock cycles, we can see the instruction make its way through the pipeline. We move to the decode stage in the second cycle (1440ns), where some important control signals are generated, such as RegDst and jump. We can also see that the branch address is calculated in this stage, as s_ID_new_pc is not equal to s_IMemAddr + 4. The execute stage does not do anything, since a jal instruction does not require the use of the ALU, and nor does the memory stage. When it enters the writeback stage, we see that the address of the instruction following the jal is selected by the mux, and that the proper register destination is selected (in this case, $ra for a jal) and written to.

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.



- In the waveform, Two cursors are placed within an iteration of BubbleSort where less than the maximum number of NOPs for our pipeline design (3 nop instructions) are utilized to avoid data hazards while executing the minimum number of instructions necessary. According to the opcode values from I-Mem in the fetch stage at 1800 ns, the instruction sequence until 1880 ns is `ori -> nop -> nop -> sll`. This `ori` corresponds to the decoded `lasw` pseudo instruction (see Example 1 below) writing the address of the data array to register `$2`. Thus, the waveform correctly displays the correct instructions, PC register updates, and required values by each instruction. Additionally, the array address 0x10010000 can be seen on the decode stage of the `lasw` instruction and later seen on the writeback stage two cycles later, showing the data hazard was avoided and the address was properly written to register `$2`.

- Including the one in the waveform above, our program contains three examples of using less than the maximum number of no-ops between data dependencies to prevent data hazards. In our specific design of the software-scheduled pipeline, we are not able to utilize less than the maximum number of no-ops for *control* hazards, as the order of branch and jump instructions in our program cannot be

adjusted in a way to enable a solution with less no-ops. Additionally, the maximum no-ops required to prevent control hazards is *four* instead of three for data hazards, as the control flow instructions take an extra cycle to propagate signals to the PC register in order to properly branch or jump to the correct next instruction.

Example 1: Data flow lines 66-69: Here the `lasw` instruction can be used as one of three instructions executed before `sll` is fetched, since `sll` has a read dependency of `$4` which is written by the `addiu` instruction. This is more efficient than using three no-ops, requiring only two.

```
addiu   $4,$2,1
lasw    $2, array
nop
nop
sll     $4,$4,2
```

Example 2: Data flow lines 113-117: Similarly to example 1, the `lasw` instruction can be used as one of three instructions executed before `sll` is fetched, since `sll` has a read dependency of `$3` which is written by the `addiu` instruction. This requires only two additional no-ops.

```
addiu   $3,$2,1
lasw    $2, array
nop
nop
sll     $3,$3,2
```

Example 3: Data flow lines 124-128: Here the `lw` and `lasw` instructions have been reordered to execute before `sll` is fetched, since `sll` has a read dependency of `$4` which is written by the first `lw` instruction. The `lw` and `lasw` don't have any dependencies, so they can execute individually and both before the `sll` dependency. Overall, this requires only one additional no-op.

```
lw      $4,12($fp)
lw      $3,0($2)
lasw  $2, array
nop
sll     $4,$4,2
```

[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

- Our reported frequency is **50.86 MHz**
- The critical path is **ID/EX Register > ALU Control Unit > ALU > EX/MEM Register**
- Given the software scheduled pipeline is a requirement (instead of implementing hardware scheduled hazard detection), the most beneficial component to optimize here would be the ALU, as it takes the most time in the critical path (15.56 ns), which is located in the Execute Stage
- Since this processor design is pipelined, or each stage has its own path each cycle, the critical path is located in the slowest STAGE. So in this case, our slowest stage is Execute with the ALU taking the longest to propagate signals. This is reasonable, as our hazard detection hardware is not in this design, although it's interesting Decode is faster despite the PC add and jump logic executing with the register file there

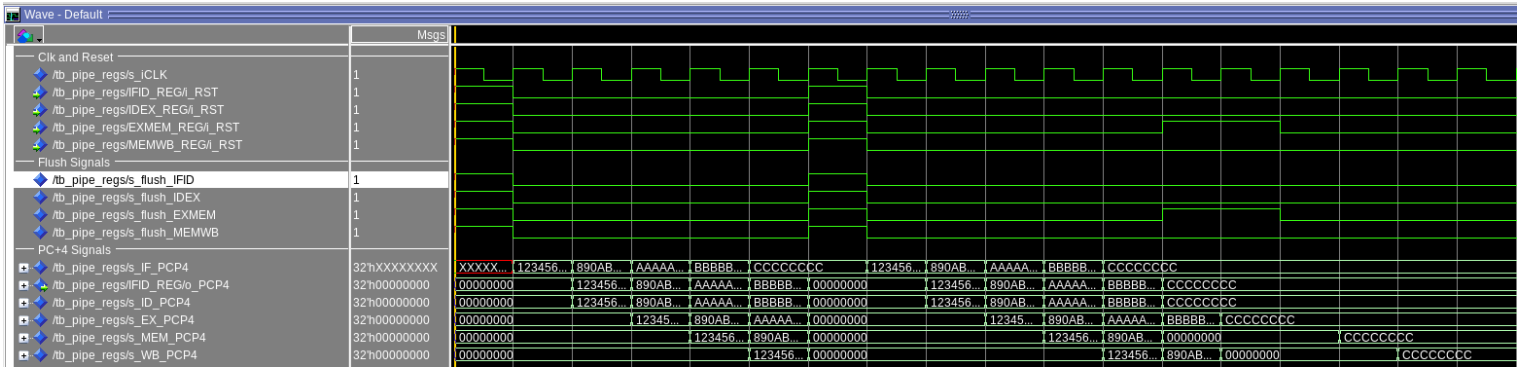
[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



A register would stall via a mux that 0's out write/ some control flow signals into its input.

It is flushed via a seperate signal that resets the entire register.

[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



[2.b.i] List which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

- Instructions produce values on **s_EX_ALUOut** signal**:** add, addi, addiu, addu, and, andi, nor, xor, xori, or, ori, slt, slti, sll, srl, sra, sw, sub, subu
- Instructions produce values on **s_DMemOut** signal: lw
- Instructions produce values on **s_EX_lui_val**: lui

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

- Instructions consume values on **s_Inst_Rd, s_Inst_Rs, s_Inst_Rt, s_dsrc1, s_dsrc2, s_Inst[10:6] (R-type):** add, addu, and, nor, xor, or, slt, sll, srl, sra, sub, subu
- Instructions consume values on **s_Inst_Rd, s_Inst_Rs, s_Inst_Rt, s_dsrc1, s_dsrc2, s_sign_ext_imm (I-type):** addi, addiu, andi, lui, lw, xori, ori, slti, sw

[2.b.iii] Generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

- Forwarding Dependencies:
  - **EX/MEM s_Inst_Rd = ID/EX s_Inst_Rs, EX/MEM s_Inst_Rd = EX/MEM s_Inst_Rt:** Forward *s_ALUOut* back to input of Execute stage of next instruction
- Stalling Dependencies (Load-use dependencies):
  - **MEM/WB s_Inst_Rd = ID/EX s_Inst_Rs, MEM/WB s_Inst_Rd = ID/EX s_Inst_Rt:** *s_DMemOut* needs stalls and a bubble in the pipeline until it can propagate to the register file

[2.b.iv] Global list of the datapath values and control signals that are required during each pipeline stage

IF/ID: PC+4, Instruction
ID/EX:

- Old: PC+4
- New: new_PC, do_branch, branchCtrl, CtrlRegWrite, RegDest, jump, memSel, ALUSrc, ALUOp, DMemWrite, halt, dsrc1, dsrc2, sign_ext_imm, Instr[25:21] (rs), Instr[20:16] (rt), Instr[15:11] (rd), Instr[15:0] (lui), Instr[10:6] (shamt), Instr[5:0] (function)

EX/MEM:

- Old: PC+4, new_PC, do_branch, branchCtrl, CtrlRegWrite, RegDst, jump, memSel, halt, dsrc2, Instr[20:16] (rt), Instr[15:11] (rd), Inst[25:21] (rs)
- New: lui_val, overflow, ALUOut

MEM/WB

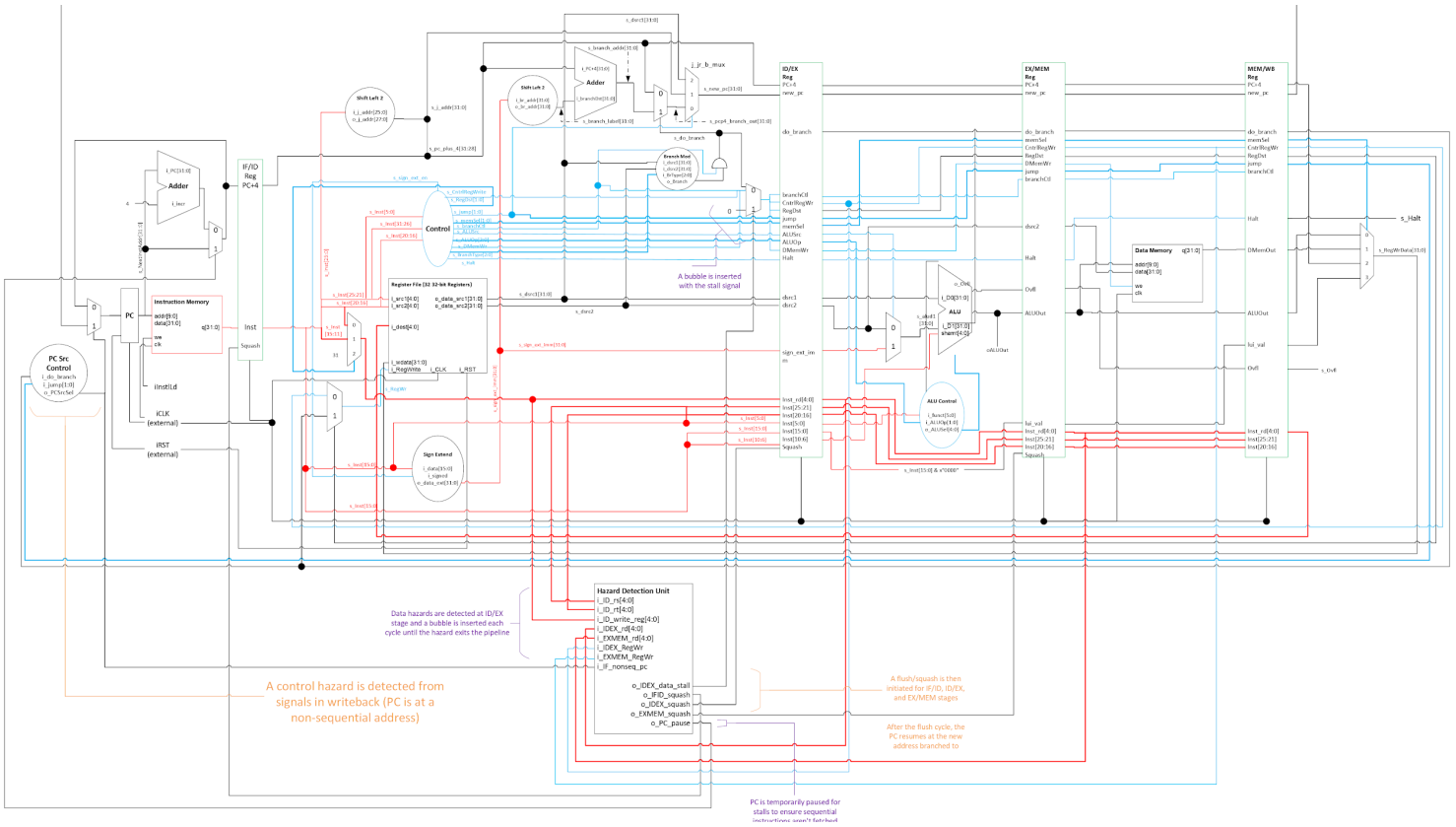
- Old: PC+4, new_pc, do_branch, branchCtrl, memSel, CtrlRegWrite, RegDst, jump, ALUOut, halt, Inst[25:21] (rs), Instr[20:16] (rt), Instr[15:11] (rd), lui_val, overflow
- New: DMemOut, s_RegWrData

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.
Non-sequential PC updates are calculated in Decode, but this new PC value is not available to use until it is in the Writeback stage for the following instructions: **beq, bne, j, jal, jr, bgez, bgezal, bgtz, blez, bltzal, bltz**. The PC fetches this new PC value in Fetch from a MUX output

[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

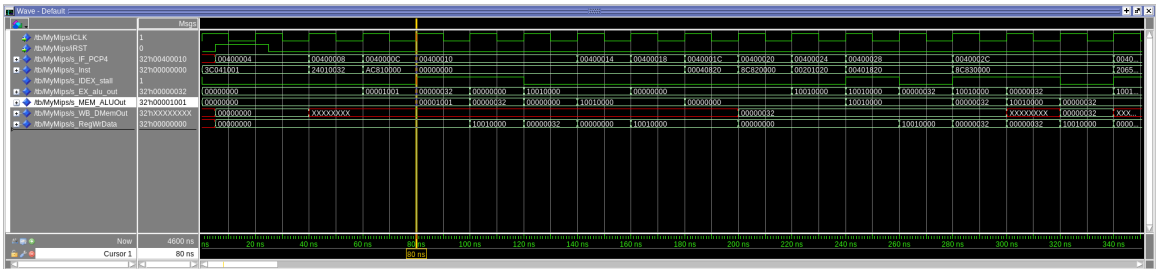
- Stages stalled: **ID/EX**
- Stages squashed/flushed: **IF/ID, ID/EX, EX/MEM**

[2.d] Implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



For better viewing of annotations, please view the additional diagram PDF included in this submission.

[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.



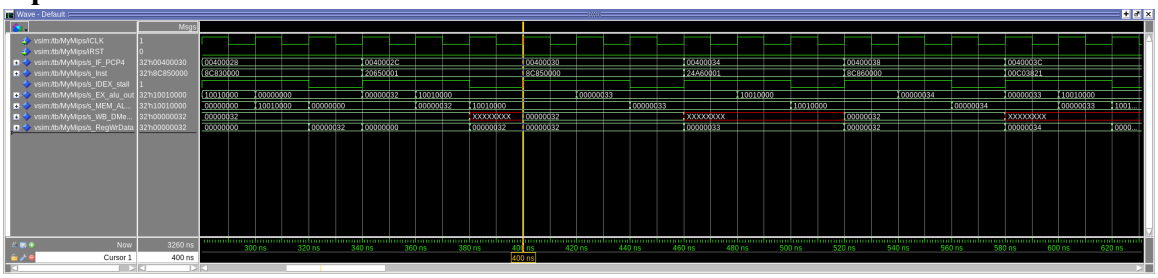
/data_hazard_tests/**data.s** - This combines tests from id-ex.s, ex-mem.s, and mem-wb.s. These tests introduce ID/EX, EX/MEM, and MEM/WB Rd and MemRead data hazards in the pipeline. The test uses a combination of data dependencies used sequentially (no instructions between the dependency), after one instruction between the dependency, and with `lw` instructions. Additionally, the dependency alternates between the rs field and rt field to ensure the hazard is detected for both fields.

In the above combined test, the output works as expected. The ID/EX stall signal goes high on the cycle detecting a hazard. In this case, the first example this occurs in MIPS is:

addiu $1, $0, 50
sw $1, 0($4)

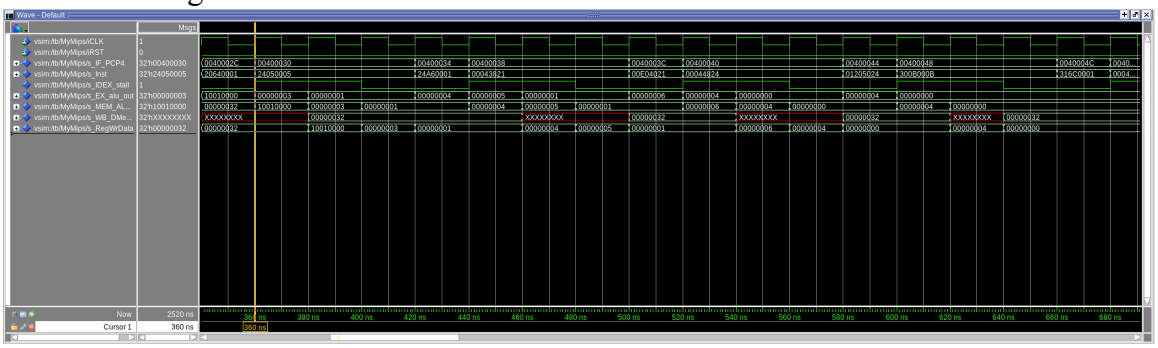
The sw instruction has a dependency on addiu writing a value to $1. Here, the PC+4 signal is paused, no instruction is fetched, and the ALU continues processing values to the right of the stalled pipeline register. The stall signal remains high until the hazard clears the pipeline and the instruction fetching resumes. Later on DMemOut, when a `lw` reads the same memory location, the value 0x32 = 50 is read, which means the pipeline properly wrote the value at the end of the instruction execution from the previous `addiu` and the dependency was properly stalled for.

Separated data hazard tests:

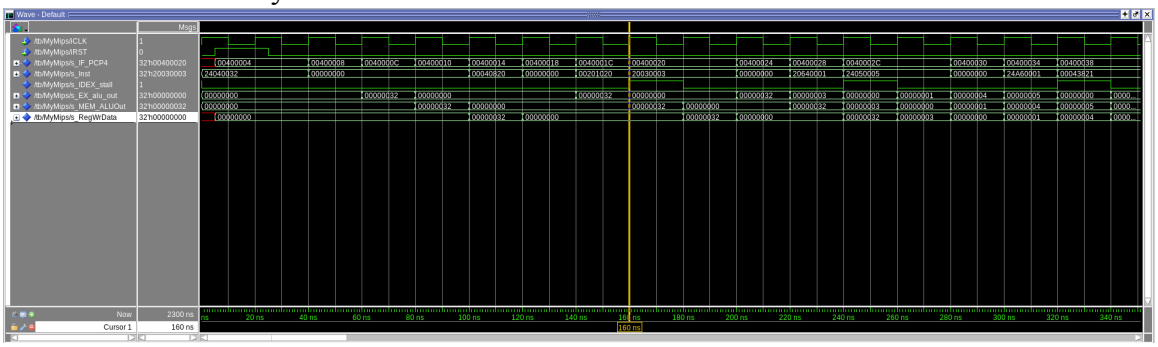


Load-use hazards - This works the same as the explanation given above

EX/MEM Rd hazards - The value written to on the previous instruction is correctly read in the following instruction from the Rs or Rt fields.

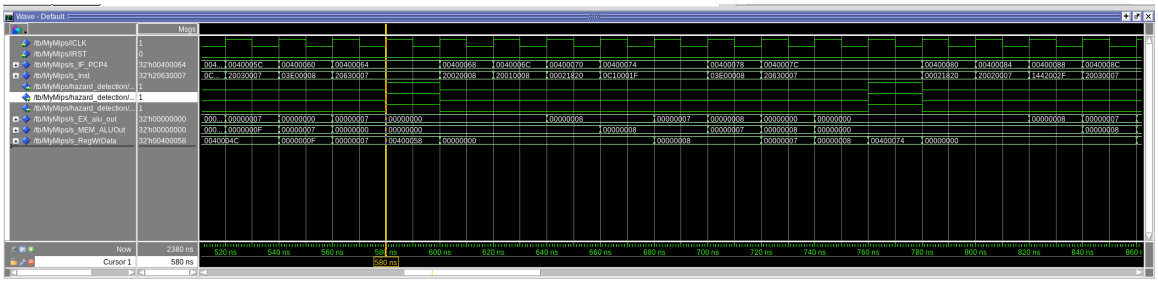


MEM/WB Rd hazards - The value written to on two instructions before the current instruction is correctly read on the current in the Rs or Rt fields.



Here, one nop is used to separate the instructions by one clock cycle, to ensure the hazard is still detected. Our output properly detects it and sets the stall signal high for a shorter amount of time than the EX/MEM test case, which is expected, since the dependency is spaced apart by two stages instead of one.

Control Hazards:



/control_hazard_tests/**control.s** - This program combines all of the branch and jump instruction individual tests into one large branching and control flow test program. These tests introduce every type of control hazard in the pipeline. The test works by using a combination of branch dependencies used sequentially (no instructions between the dependency), after one instruction in between the dependency, and two instructions between. Additionally the use of the dependency alternates between the branch condition itself and the instructions after the branch instruction. This exhaustively tests the pipeline's ability to flush itself or remove instructions no longer valid if a branch or jump is taken.

In the above combined test, the output works as expected. The flush signal for each pipeline register (ID/EX, EX/MEM, MEM/WB) goes high on the cycle detecting a hazard. In the above waveform screenshot, the example where this happens is the following in MIPS:

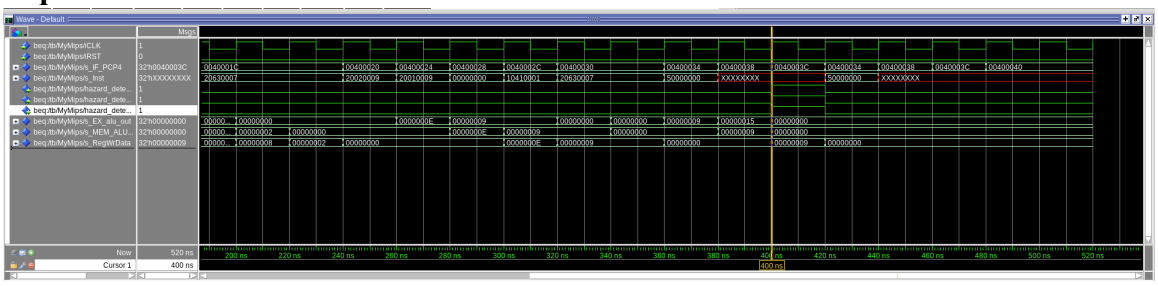
addi $2, $0, 7

jal two

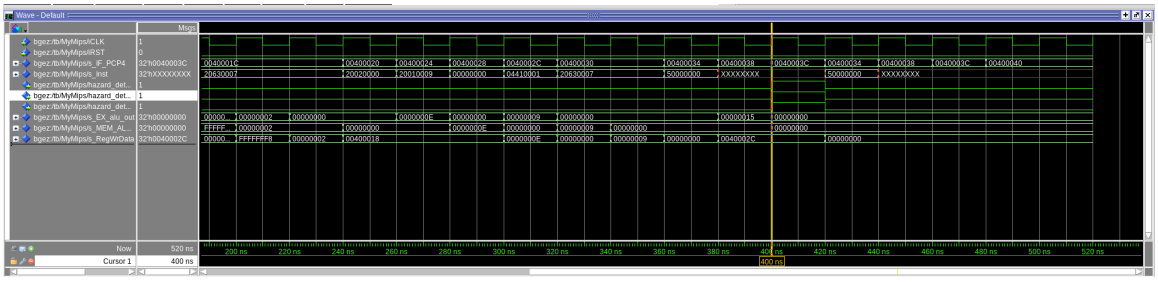
Jal is an instruction that always jumps when executed. When the instruction is brought into fetch, the pipeline cannot act on the address to jump to until the writeback stage, since the $ra register must be written to and the PC must be updated. During that time the pipeline continues fetching instructions, causing a hazard of misinformation in the pipeline (additionally the PC has still been incrementing by 4 as seen in the screenshot). Here, the PC+4 signal is paused, no instruction is fetched, and the ALU continues processing values to the right of the stalled pipeline register. Once it reaches the writeback stage, the pipeline asserts the flush signals, wiping out the unneeded instructions after the jump, up until the jump clears and the PC matches the new line the program jumped to. After, the flush desserts and the pipeline resumes. In writeback, Immediately on RegWriteData, the jal instruction writes to $ra the current PC address to link back to if a jr $ra is executed later. As seen in the waveform, each of these occurrences means the pipeline properly stalled for the dependency and properly flushed any unnecessary instructions following the jump execution. These correct flushes also occur for every other control flow instruction included below:

Separated control hazard tests:

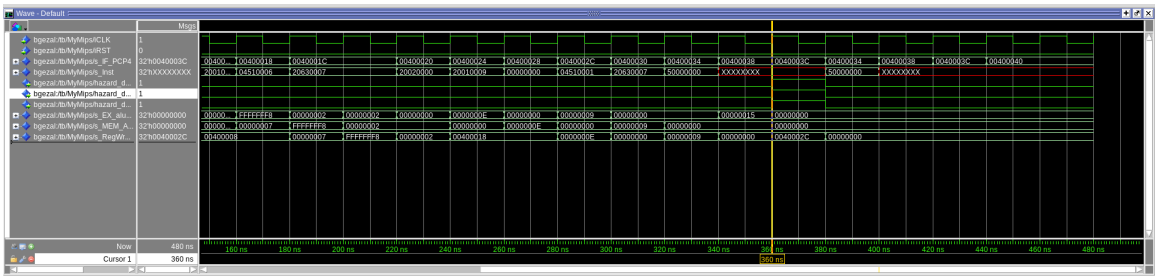
beq



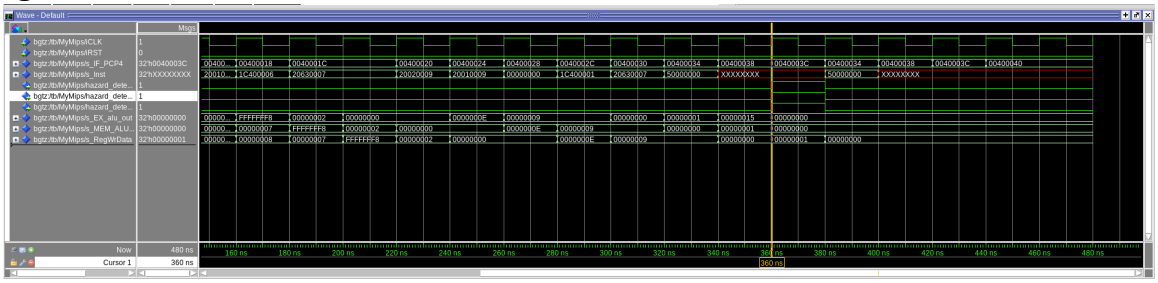
bgez



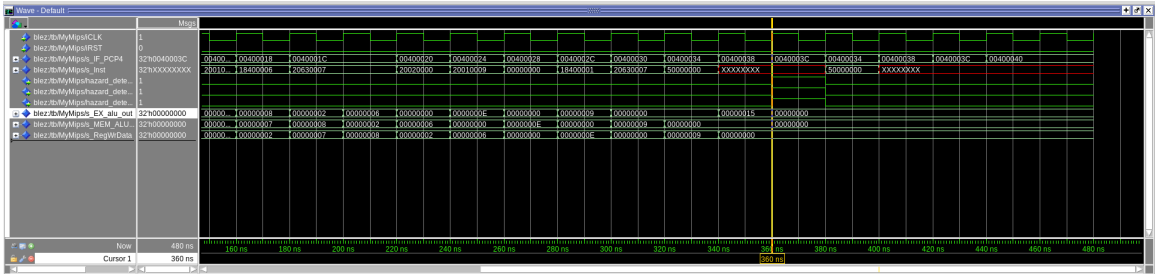
bgezal



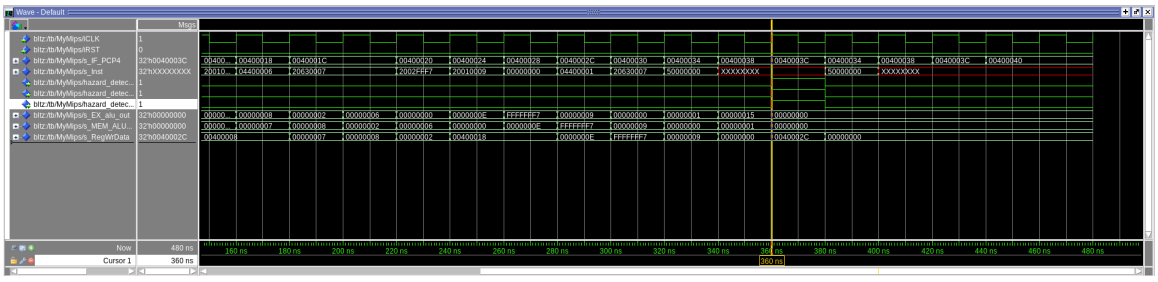
bgtz



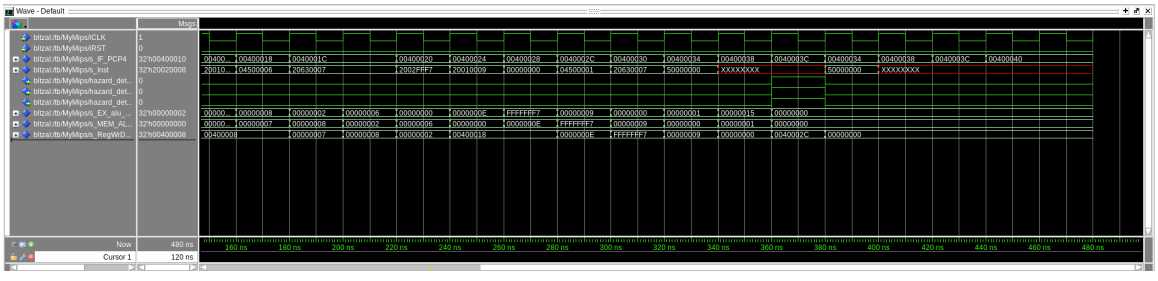
blez



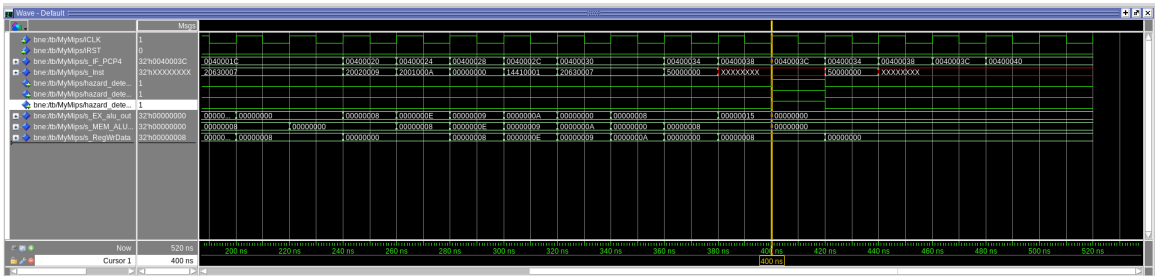
bltz



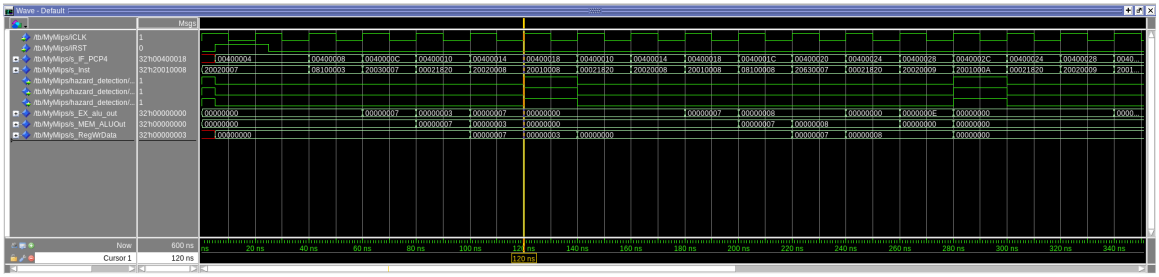
bltzal



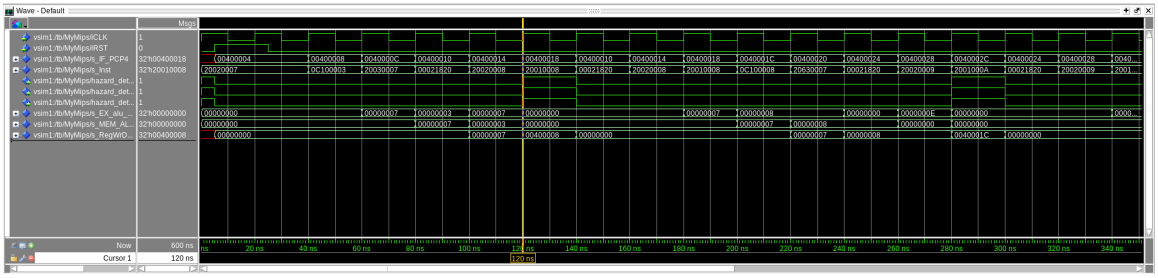
ne



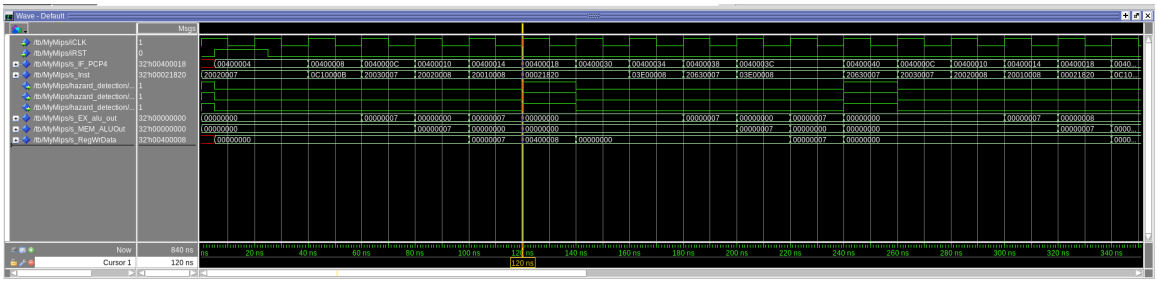
j



jal



jr



All instructions with non-sequential PC updates are seen flushing the pipeline when a branch is taken. All of these individual test cases have similarly formatted test programs to clearly identify when the branch is taken. The branch is taken on the third occasion of a branch or jump instruction in each test program.

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

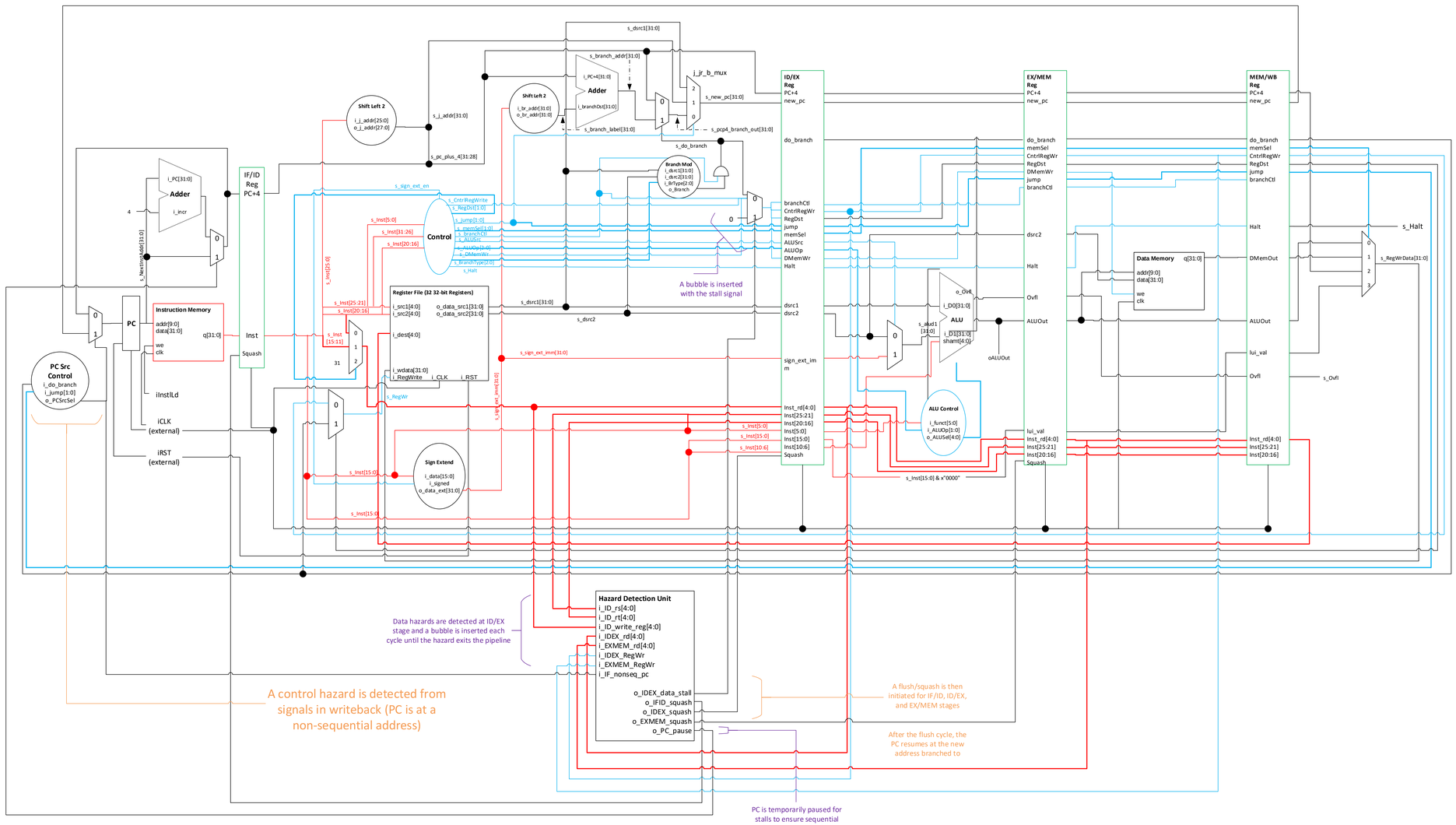
Processor Control and Hazard Signals - TermProj_3_02 /Data Hazard Test Program

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Processor Control and Hazard Signals - TermProj_3_02 /Control Hazard Test Program

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

- Our reported frequency is **46.24 MHz**
- The critical path is **Register File > Branch Control Unit > Jump/Jump Register/Branch MUX > ID/EX Register**
- For our hardware scheduled pipeline the most beneficial component to optimize here would be the Register File, as it takes the most time in the critical path (6.07 ns), which is located in the Decode stage
- Since this processor design is pipelined, or each stage has its own path each cycle, the critical path is located in the slowest STAGE. So in this case, our slowest stage is Decode with the Register File taking the longest to propagate signals. This is reasonable, as our hazard detection hardware is located in the Decode stage (branch control and hazard detection units), although it's interesting the register file is still the slowest component and no longer faster than the ALU, despite the ALU not changing. This is likely due to us changing the register file to accept writes on the negative edge of the clock instead of the positive edge


s_dsrc1[31:0]
s_branch_addr[31:0]
i_PC+4[31:0]
Adder
i_branchOst[31:0]
j_jr_b_mux
s_new_pc[31:0]
Shift Left 2
i_j_addr[25:0]
o_j_addr[27:0]
s_j_addr[31:0]
Shift Left 2
i_br_addr[31:0]
o_br_addr[31:0]
s_branch_label[31:0]
s_pcp4_branch_out[31:0]
s_pc_plus_4[31:28]
s_do_branch
Branch Mod
i_dsrc1[31:0]
i_dsrc2[31:0]
i_BrType[2:0]
o_Branch
i_PC[31:0]
Adder
i_incr
4
s_NextInstrAddr[31:0]
IF/ID
Reg
PC+4
s_sign_ext_en
s_CntrlRegWrite
s_RegDst[1:0]
s_jump[1:0]
s_memSel[1:0]
s_branch[1:0]
s_ALUSrc
s_ALUOp[3:0]
s_DMemWr
s_BranchType[2:0]
s_Halt
Control
s_Inst[5:0]
s_Inst[31:26]
s_Inst[20:16]
s_Inst[25:0]
A bubble is inserted
with the stall signal
Register File (32 32-bit Registers)
i_src1[4:0]
i_src2[4:0]
o_data_src1[31:0]
o_data_src2[31:0]
i_dest[4:0]
i_wdata[31:0]
i_RegWrite
i_CLK
i_RST
s_RegWr
s_dsrc1[31:0]
s_dsrc2
s_Inst[25:21]
s_Inst[20:16]
s_Inst[15:11]
s_sign_ext_imm[31:0]
s_sign_ext_imm[31:0]
Instruction Memory
addr[9:0]
data[31:0]
q[31:0]
we
clk
PC
PC Src
Control
i_do_branch
i_jump[1:0]
o_PCSrcSel
iInstLd
iCLK
(external)
iRST
(external)
Inst
Squash
s_Inst[15:0]
Sign Extend
i_data[15:0]
i_signed
o_data_ext[31:0]
s_Inst[15:0]
s_Inst[15:0]
ID/EX
Reg
PC+4
new_pc
do_branch
branchCtl
CntrlRegWr
RegDst
jump
memSel
ALUSrc
ALUOp
DMemWr
Halt
dsrc1
dsrc2
sign_ext_im
m
inst_rd[4:0]
inst[25:21]
inst[20:16]
inst[5:0]
inst[15:0]
inst[10:6]
Squash
s_Inst[5:0]
s_Inst[15:0]
s_Inst[10:6]
s_Inst[10:6]
Inst[10:6]
Squash
s_Inst[15:0] & x"0000"
o_Ovfl
i_D0[31:0]
ALU
i_D1[31:0]
shamt[4:0]
s_aluD1
[31:0]
oALUOut
ALU Control
i_funct[5:0]
i_ALUOp[1:0]
o_ALUSel[4:0]
EX/MEM
Reg
PC+4
new_pc
do_branch
memSel
CntrlRegWr
RegDst
DMemWr
jump
branchCtl
Halt
Ovfl
ALUOut
lui_val
Inst_rd[4:0]
Inst[25:21]
Inst[20:16]
Squash
Data Memory
addr[9:0]
data[31:0]
we
clk
q[31:0]
MEM/WB
Reg
PC+4
new_pc
do_branch
memSel
CntrlRegWr
RegDst
jump
branchCtl
Halt
DMemOut
ALUOut
lui_val
Ovfl
Inst_rd[4:0]
Inst[25:21]
Inst[20:16]
s_Halt
s_RegWrData[31:0]
s_Ovfl
Hazard Detection Unit
i_ID_rs[4:0]
i_ID_rt[4:0]
i_ID_write_reg[4:0]
i_IDEX_rd[4:0]
i_EXMEM_rd[4:0]
i_IDEX_RegWr
i_EXMEM_RegWr
i_IF_nonseq_pc
o_IDEX_data_stall
o_IFID_squash
o_IDEX_squash
o_EXMEM_squash
o_PC_pause
Data hazards are detected at ID/EX stage and a bubble is inserted each cycle until the hazard exits the pipeline
A control hazard is detected from signals in writeback (PC is at a non-sequential address)
A flush/squash is then initiated for IF/ID, ID/EX, and EX/MEM stages
After the flush cycle, the PC resumes at the new address branched to
PC is temporarily paused for stalls to ensure sequential instructions aren't fetched

CprE 381: Computer Organization and Assembly-Level Programming

Project Part 3: Performance Analysis and Report

Team Members: Anthony Manschula

Henry Shires

Project Teams Group #: TermProj_3_02

Introduction

Our project encompassed a total of three processor designs. The first was a single-cycle design, the second a software-scheduled pipelined design, and the third a hardware-scheduled pipelined design. We faced many challenges, both organizational and technical, but were able to overcome those issues by managing our workload using Git source control, Gitlab issues, as well as various resources from class and the internet. Each design was synthesized after successful testing with software programs provided, and a maximum frequency and critical path were extracted from the synthesis results. These performance metrics recorded from synthesis allowed us to perform competitive analysis between our designs to see which ones perform the best, all fully outlined in this report.

Benchmarking

Table 1: Single-Cycle Benchmarks

Test	Inst Count	Total Cycle Count	CPI	Max Cycle Time	Total Execution Time
Synthetic Inst Test	41	41	1.0	4.46E-8 sec	1.82us
Grendel	2116	2116	1.0	4.46E-8 sec	94.37us
Bubble Sort	1033	1033	1.0	4.46E-8 sec	46.07us

Table 2: Software-Pipelined Benchmarks

Test	**Inst Count**	**Total Cycle Count**	**CPI**	**Max Cycle Time**	**Total Execution Time**
Synthetic Inst Test	90	138	1.53	1.97E-8 sec	2.71us
Grendel	5519	6551	1.19	1.97E-8 sec	129.05us
Bubble Sort	3168	3340	1.05	1.97E-8 sec	65.80us

Table 3: Hardware-Pipelined Benchmarks

Test	**Inst Count**	**Total Cycle Count**	**CPI**	**Max Cycle Time**	**Total Execution Time**
Synthetic Inst Test	41	103	2.51	2.16E-8 sec	2.22us
Grendel	2116	5144	2.43	2.16E-8 sec	111.11us
Bubble Sort	1033	2439	2.36	2.16E-8 sec	52.6us

Performance Analysis

To start, the relative performance numbers were roughly about what we expected them to be. We used a combination of the test framework output and the basic performance equation to obtain the numbers in the above charts. Since the framework already gives us instruction count, cycle count, and CPI, no math was necessary there. We obtained a critical path from our synthesis for each design, which also allowed us to obtain the theoretical maximum processor frequency. The maximum cycle time was found by taking $\frac{1}{f_{max}}$. Since we already know the total cycle count as given by the provided tool flow output, we can simplify the performance equation down to

$$Execution\ Time = Total\ Cycle\ Count \cdot Max\ Cycle\ Time$$

This will give us our final metric of performance that we can then use to compare each design.

We predicted our single-cycle design to perform middle-of-the-road before beginning part two since we had some background knowledge from class regarding how it functioned. Because all

instructions must traverse all processor components in one cycle, maximum clock speed is severely hindered by any components that might have significant design issues that would cause a considerable amount of latency. For this reason, it can be seen that the clock frequency of the single–cycle design is significantly lower than the two other designs, being roughly 2x lower than our highest, the software (SW) pipeline. Despite the low clock frequency, however, the single-cycle nature of the first design meant the total execution time of each program was instead the lowest for this design relative to the other two. The following paragraphs include insight into this nuance in our design.

Our software-pipelined processor was the second design we completed. Initially, we expected that this processor would have the worst overall performance, considering that the penalty for a data dependency or control hazard instruction in the pipeline went from zero to three cycles. However, due to the pipelined nature of this design, our clock speed increased drastically, up from 22.49Hz to ~50MHz. This clock frequency increase stems from the pipelined processor's design splitting component signal propagation into multiple cycles. The clock frequency, rather than resulting from *all* of the processor components' latencies in one cycle, only propagates through individual stages of the processor per cycle and can be determined by the pipeline stage with the longest individual latency. However, using our knowledge of the performance formula, we deduced that, despite an over 2x jump in clock speed, the overall execution time would likely still be worse due to the inclusion of many necessary stalls. That reality is reflected in Table 2 on page 2; the overall execution time increased substantially from the single-cycle processor, or about 1.43x on average across the three tests.

Our last challenge to tackle was the hardware-pipelined implementation. While from a design organization standpoint, there were not many changes to be made. However, there were a few challenges regarding improving performance and ensuring correct functionality. The first improvement was a negative-edge triggered register file, automatically allowing us to remove one stall cycle between a read-after-write (RAW) dependency in the pipeline. Our next task involved data and control hazard detection, which, while not directly improving performance, would allow the processor to run a normal, unmodified MIPS assembly file (without manually including NOP instructions). Implementing the

necessary components for this resulted in a slower critical path, which brought down our maximum clock speed by about 3.5MHz. Unfortunately, due to time constraints with other projects from other classes, neither of us could devote the time to developing a reliable data forwarding component. This setback meant that our hardware pipeline could not improve as much performance over the software-scheduled pipeline as we had hoped since we still had to stall for two cycles for data hazards and flush several stages for control flow instructions. If we had implemented data forwarding, our CPI would have improved enough to the point where it nearly matched or beat the single-cycle design in total execution time. Additionally, while it wasn't expected from many groups to implement, having branch prediction for control hazards would also greatly improve our CPI, reducing the number of flushes needed for branching instructions. There is one quick note to be made regarding the control-flow hazard behavior of our design, however. It had not occurred to us that we could pull control flow information from several stages and resolve control hazards that way, and thus we ended up proceeding with a suboptimal component design in that regard. Currently, our design passes the final resolved branch/jump address through the entire pipeline from the decode stage (where it's calculated) back to the PC-source mux in the fetch stage. This results in a much harsher penalty (3 cycles) since we now have to flush the IF/ID, ID/EX, and EX/MEM pipeline registers whenever we take a branch or a jump instruction.

Concerning relative differences between tests, all three tests saw roughly the same performance variation from design to design, assuming that the single-cycle processor is the baseline. Looking at the assembly files, this is likely because there are many read-after-write dependencies in each of the tests, so we would expect to see a similar amount of stalling necessary across the three tests. This detail would explain why the relative difference is similar.

Software Optimization

Reducing data dependencies via reordering instructions or varying register usage more heavily would positively affect the performance of the software-pipelined processor by lowering the number of

stall cycles necessary. This reduction would increase the effective CPI of the design since we would have to stall less per meaningful instruction.

Hardware Optimization

1. **Single-Cycle**

   The optimization that would best improve this design is the ALU, specifically the barrel shifter. According to our synthesis results, the barrel shifter accounts for about 4ns of latency in total. If we could optimize each stage of the barrel shifter to remove even one nanosecond of latency, our clock speed would increase by about 450 kHz.

2. **Software Pipeline**

   Optimizing the critical path of the software-scheduled design would improve our clock speed, which would decrease our overall execution time. Our critical path was in the Execute stage for this design and was mostly due to the design of the ripple-carry adder in the ALU. This latency could be reduced considerably if this design were replaced with optimized components such as a carry-look-ahead adder.

3. **Hardware Pipeline**

   Data forwarding was a major improvement we could not implement on the hardware-pipelined version in time. This improvement would have theoretically allowed us to eliminate both remaining stall cycles from certain read-after-write dependency situations, bringing down our CPI by a large margin and potentially allowing us to break even with or surpass the performance on the single-cycle design.

Performance of Programs: It Depends

Since we know that the single-cycle design's performance already surpasses the hardware-pipelined version's performance in instances with lots of data dependencies, we could write a program that only accesses one or two registers or has many control flow instructions. Although the

single-cycle version has a slower clock speed, the number of stall cycles necessary from the pipeline will outweigh any potential benefit its higher clock speed will have. Conversely, a lengthy program with few control flow instructions or data dependencies would perform better on the pipelined version. The length of the program is important because, assuming no or few stalls are present, a longer program will bring your CPI closer to the ideal 1.0. And since our CPI is nearly the same as the single-cycle processor and we have a higher clock speed, the pipelined processor would outperform the other.

To determine exactly where the break-even point is between a program running better on the single-cycle versus hardware pipeline design, we can calculate the maximum number of average stalls per instruction allowed using the break-even execution time between the two processors. The execution time formula can be modified to determine the average stalls required per instruction by converting CPI to 1.0 ideal CPI plus the average number of stall cycles for each instruction. This new equation can be solved for the maximum number of stall cycles allowed per instruction before the program is faster on the single-cycle design.

$$Execution\ Time\ =\ Total\ Cycle\ Count\ \cdot\ Max\ Cycle\ Time$$

$$ETsc\ =\ EThw$$

$$Instr\ *\ CPI\ *\ Cycle\ Time\ =\ Inst\ *\ (1.0\ +\ (avg\ stalls\ per\ instruction))\ *\ Cycle\ Time$$

$$1\ *\ 4.46E\ -\ 8\ sec\ =\ (1\ +\ avg)\ *\ 2.16E\ -\ 8\ sec$$

$$avg\ =\ 1.06\ or\ about\ 1\ stall\ cycle\ per\ instruction\ allowed$$

Challenges

Our team faced multiple challenges during all three processor implementations' design, building, and testing phases. We struggled with non-technical challenges, such as finding suitable times throughout the week to work together on design and debugging, and technical challenges, like writing proper VHDL to compile, synthesize, and integrate components in our final design.

First, we struggled to work on the project together due to our busy schedules and multiple class commitments. Initially, during our single-cycle design, we found a rhythm in meeting twice weekly for three hours, drawing solutions, and discussing ideas we could build individually and integrate later.

However, as our schedules diverged and deadlines approached, we realized our pace on the project needed to be faster. To pick up the slack, we both went overtime individually to make up for the difference, but this lack of togetherness ultimately caused discrepancies in understanding our design and implementation of our processor. We had to find other ways to share ideas or time manage our other classes to work at a good speed on our project. Ultimately, we successfully balanced our workload, completing all three parts of the project during the due weeks. To solve this, we clearly outlined the remaining tasks and followed the task schedule in the initial team contract we established. We also utilized Gitlab project flow tools like issues and milestones to track progress, just like practices in software development. These minor additions helped us communicate the project's elements that needed completion, who was doing what, and when.

Second, we had difficulties writing VHDL to comply with structural architecture syntax guidelines required by the top-level design. While this architecture best synthesizes for the FPGA, we had difficulty considering our design regarding logic components instead of general programming statements (process, if/then, looping, etc.). Our backgrounds involve high-level software projects, so switching mindsets to low-level hardware projects was difficult. Since we successfully got all three designs to work, our process involved occasional paired programming sessions in coming up with new ideas or solutions in structural to increase our chances of first-try synthesis. Additionally, referencing online resources, lab resources provided, and other students' solutions without copying helped us think about each component differently, ultimately leading us to an optimized result.

Third, we spent a long time painfully debugging our single-cycle and hardware-scheduled pipeline designs. These designs were extremely complicated and required hundreds of lines of new VHDL to meet the project requirements. Having little understanding of what to implement and proceeding to program it in VHDL caused bugs all over our codebase. Additionally, being rushed for time, we neglected to start with compiling drawings, spreadsheets, and tables to help us brainstorm and document necessary signals and relationships, instead jumping straight into code, likely causing further unorganized development. Since we took longer on the software-scheduled design than we should have

(implementing the registers and running test programs), we had less time to concentrate on building a quality hardware-scheduled design with multiple optimization techniques (data forwarding and branch prediction). To combat extensive debugging requirements, we found sacrifices to make in our schedules and support each other through difficult sessions in a lab to find and correct issues. By doing it together, even if only one person could type at a time, having two minds on the problem was far better than one, finding discrepancies in thinking and simple mistakes or typos way quicker. This change in our routine ultimately saved us the headache of submitting a processor late, even if we didn't get to implement all the features we had hoped for.